

学 号 :

22200107210

成绩:

苏州科技大学

毕业设计（论文）

题 目 基于检索增强生成的法律智能问答

系统设计与实现

性 质: 毕业设计☒

毕业论文☐

院 系: 电子与信息工程学院

专 业: 计算机科学与技术

年 级: 2022 级

姓 名: 韩守坤

指导教师: 王飞

二〇二六

基于检索增强生成的法律智能问答系统 设计与实现

摘 要

随着我国法治建设的不断深入，公众对便捷、精准法律咨询服务的需求日益增长。传统法律检索工具主要依托关键词匹配技术，而新兴的大语言模型则展现出强大的自然语言理解能力。然而，通用模型在处理法律业务时，仍面临时效性滞后等技术壁垒。目前，国内外研究现状多集中于垂直领域模型的微调以及检索增强生成技术的初步应用，探索如何将大规模预训练知识与特定行业私有库相结合。针对上述背景，本课题开展了基于检索增强生成技术的法律智能问答系统的研究与实现。本文设计并实现了一套多源协同检索增强架构，选用国产高性能 Qwen 系列大模型作为生成中枢，通过对核心法规进行知识解构，构建了高维向量索引的本地向量数据库。在预处理阶段，应用递归字符切分算法结合语义嵌入模型，有效处理了法律条文层级复杂的信息特征，实现了法条级的精准召回。同时，系统集成实时搜索 API 以补充数据，实现了本地法条与最新司法解释、社会案例的异构数据协同。在交互逻辑层面，通过提示词工程对模型进行强约束，增强了输出的可解释性。根据本系统的实验性结果表明，本文为大语言模型法律咨询时的时效性滞后、回答缺乏可解释性的问题提供了可行的解决方案，在确保响应时效与检索精度达到设计要求的同时，提升了回复的专业性与可靠性，为大语言模型在公众法律咨询服务的应用提供了参考。

关键词 检索增强生成；大语言模型；法律智能问答；向量数据库

Design and Implementation of a Legal Intelligent Question Answering System Based on Retrieval-Augmented Generation

Abstract

With the continuous advancement of China's rule of law, there is a growing public demand for convenient and accurate legal consultation services. Traditional legal retrieval tools primarily rely on keyword matching technology, whereas emerging Large Language Models (LLMs) demonstrate robust natural language understanding capabilities. However, universal models still face technical barriers such as time-lagged information when processing legal affairs. Currently, research both domestically and abroad focuses on fine-tuning domain-specific models and the preliminary application of Retrieval-Augmented Generation (RAG) technology, exploring ways to integrate large-scale pre-trained knowledge with industry-specific private repositories. Against this background, this project conducts research on and implements a legal intelligent Q&A system based on RAG technology. This thesis designs and implements a multi-source collaborative retrieval-augmented architecture, selecting the high-performance domestic Qwen series LLM as the generative core. By deconstructing core regulations, a local vector database with high-dimensional vector indexing was constructed. In the pre-processing stage, a recursive character splitting algorithm combined with a semantic embedding model was applied to effectively process the complex hierarchical information of legal provisions, achieving precise clause-level recall. Meanwhile, the system integrates a real-time search API to supplement data, achieving synergy between local statutes and heterogeneous data such as the latest judicial interpretations and social cases. At the interaction logic level, prompt engineering is utilized to impose strong constraints on the model, enhancing the interpretability of the output. According to the experimental results of this system, it is demonstrated that this thesis provides a feasible solution to the issues of

time-lagged information and lack of interpretability in LLM-based legal consultations. While ensuring that response timeliness and retrieval accuracy meet design requirements, the system improves the professionalism and reliability of responses, providing a reference for the application of LLMs in public legal consultation services.es with clear citations, offering substantial engineering reference value.

Keywords Retrieval-Augmented Generation; Large Language Model; Legal Intelligent Question-Answering; Vector

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 研究现状	1
1.2.1 国内外发展状况	2
1.2.2 法律检索技术的演进	2
1.2.3 检索增强生成的技术优化趋势	2
1.3 本文研究内容	3
1.4 论文组织结构	4
第 2 章 相关理论基础	5
2.1 大语言模型	5
2.1.1 大语言模型概述	5
2.1.2 Qwen 系列大语言模型	6
2.2 检索增强生成的核心原理	6
2.2.1 RAG 技术概述	6
2.2.2 RAG 的基本流程	6
2.3 提示词工程	7
2.3.1 提示词工程概述	7
2.3.2 提示词设计原则与本系统的应用	7
2.4 向量表征与数据预处理技术	8
2.4.1 文本嵌入原理	8
2.4.2 相似度度量算法	8
2.4.3 法律长文本的递归切分策略	9
2.5 系统架构技术	9
2.5.1 前端开发框架	9
2.5.2 后端开发框架	10
2.5.3 向量数据库	10
2.5.4 关系型数据库	10
2.6 本章小结	10
第 3 章 系统需求分析	11
3.1 需求概述	11
3.2 业务功能分解	11
3.3 功能性需求分析	12
3.3.1 智能法律问答功能需求	12
3.3.2 知识库管理功能需求	13
3.3.3 会话管理功能需求	14
3.4 非功能性需求	15
3.4.1 准确性与权威性要求	15
3.4.2 性能性要求	15
3.4.3 可靠性与稳定性要求	16

3.5 本章小结	16
第 4 章 系统设计	17
4.1 系统总体架构设计	17
4.2 核心业务流程设计	18
4.3 功能模块详细设计	22
4.3.1 知识库管理模块设计	22
4.3.2 智能法律问答模块设计	24
4.3.3 会话管理与交互模块设计	29
4.4 数据库设计	31
4.4.1 核心功能表	32
4.4.2 辅助功能与后台统计表	35
4.5 本章小结	38
第 5 章 系统实现	39
5.1 系统登录与注册功能实现	39
5.2 知识库管理模块实现	40
5.3 智能法律问答模块实现	42
5.4 会话管理与交互模块实现	44
5.5 管理员后台统计可视化功能实现	47
5.6 本章小结	49
第 6 章 系统测试	50
6.1 测试方案设计	50
6.2 功能测试结果	50
6.3 效果评估	52
6.3.1 回答对比	52
6.3.2 引用准确率评估	54
6.4 非功能测试结果	54
6.4.1 性能测试	54
6.4.2 安全性测试	55
6.4.3 可维护性测试	57
6.5 本章小结	58
第 7 章 总结与展望	59
7.1 研究总结	59
7.2 未来展望	59
致 谢	60
参 考 文 献	61
附录 A 译文	63
附录 B 外文原文	64

第 1 章 绪论

1.1 研究背景与意义

目前，我国法治建设正不断深入，社会公众对法律咨询的需求日益增长，人工智能与我国的司法体系也在不断融合^[1]。但是法律法条繁多且专业性较强^[2]，传统的法律检索工具大多是依赖关键字匹配，比较难真正理解用户的意图^[3]，这就进一步导致了普通用户难以获取准确的法律建议。

近年来，以 Transformer 架构为基础的大语言模型（Large Language Model, LLM）展现了强大的自然语言处理能力^[4]。而在法律这一严谨的领域，原生大模型就面临着如“事实幻觉”、知识更新滞后以及无法直接引用具体内容等问题^[5]。为了解决这些问题，检索增强生成（Retrieval-Augmented Generation, RAG）技术出现^[6]。它通过先从可靠知识库中检索相关片段，将相关片段用来引导模型生成答案，从而提升了回答的准确性与可解释性^[7]。

本研究主要针对相法律咨询的过程中存在的专业性较强等问题，尝试借助相关技术手段进行改进。研究过程中，我们结合 RAG 架构的特点，对其在法律领域中的应用进行了分析，通过语义嵌入模型实现更精确的匹配。与此同时，我们在系统设计时也同时关注了实际使用需求，希望能够帮助用户以较低成本获取基础法律咨询服务。当用户使用白话的方式提出问题后，系统能够结合相关法律条文生成对应回答，使法律信息的获取方式更加便捷，也为大模型技术在法律咨询方向的落地提供了一定参考^[8]。

1.2 研究现状

当前，法律智能化研究正经历从“规则驱动”向“数据驱动”，再到“大模型驱动”的范式演进。早期的法律信息检索依赖关键词匹配，虽然保证了检索的查准率，但缺乏对法律语义的深层理解。随着 Transformer 架构^[4]及预训练大语言模型（LLM）的爆发式增长，利用生成式人工智能处理法律文本成为学术界与工业界的热点。然而，法律领域对严谨性、时效性有极高要求，如表 1-1 所示，传统工具和原生大模型在应用中暴露出诸多问题^{[5][9]}。针对上述痛点，研究重点慢慢可转向更为高效、可控的检索增强生成（RAG）技术^{[6][10]}。

表 1-1 传统检索与原生大模型在法律应用中的问题

技术维度	现有方法	主要问题
语义理解	关键词匹配	难以处理自然语言意图
时效性管理	离线预训练	知识更新滞后
知识边界	参数化知识生成	存在“事实幻觉”

1.2.1 国内外发展状况

在国际研究领域，大语言模型（LLM）在法律科技（Legal Tech）中的应用经历了从数字化到自动化，再到目前深度人工智能集成的演进过程。尽管 Transformer 架构的模型在序列转导任务中表现卓越，但原生模型仍然会出现“幻觉”^[5]。RAG 技术自 2020 年由 Lewis 等人提出以来^[6]，因其能够动态检索外部实时知识并增强生成结果的可解释性，在国外法律科技领域得到了应用^[11]。而国内在法律大模型及 RAG 领域的起步虽然稍晚，但近年来在中文语境适配和信创生态建设方面取得了显著进展。其中以北京大学团队开发的 ChatLaw 为代表^[12]。

1.2.2 法律检索技术的演进

法律文本检索是法律问答中比较关键的一部分。传统的法律检索工具很多时候还是依赖关键词匹配，比如统计某些词出现的情况，再根据关键词进行查找。

不过，在实际法律咨询场景里，用户的表达方式和法律条文中的专业术语往往并不完全一致。很多用户会用比较口语化的方式描述问题，而法律文本本身表达相对正式。如果只依赖字面匹配，在面对一些表述模糊或者说说法不一致的问题时，就可能出现检索不到相关内容的情况

近年，研究现状显示，检索机制正从普通的关键词匹配向基于向量数据库（Vector Database）的语义匹配检索来演进^[7]。而通过嵌入模型（Embedding Model）可以将大量的法律条文转化为高维的向量并存储于向量数据库中，在后面的检索匹配时，就可以实现基于向量空间上的相似度的语义匹配^[13]。这种方式能提升系统对白话问题的理解能力。同时，向量数据库的索引机制也为处理《民法典》、《刑法》等大规模法律条文的时候提供了底层支撑^[14]。

1.2.3 检索增强生成的技术优化趋势

针对大语言模型在法律等专业领域存在的问题，结合向量数据库的 RAG 架构被广泛认为是目前缓解该现象的有效路径之一^[11]。目前主要聚焦于检索质量

提升与生成逻辑约束两个维度：

- (1) 检索端的精细化处理：在数据预处理阶段，传统的固定长度分块策略易导致法律条文的前后语义有跳跃。为此，使用递归字符切分（Recursive Character Splitting）算法，来提升检索效率^[15]。此外，通过嵌入模型（Embedding Model）将分块后的文本转化为高维向量，利用向量数据库来实现语义的相似度上的匹配，已成为提升检索准确率的主流手段^[7]。
- (2) 生成端的逻辑约束：在模型生成阶段，如何引导大模型进行约束的生成是研究的重点之一。通过提示工程（Prompt Engineering）构建相关指令能引导模型在处理法律咨询时遵循着先检索、后回答的逻辑^[16]。此外，为了增强结果的可解释性，在 Prompt 中加入让大模型进行强制性的引用约束，让大模型在输出结果中标注好所引用的法律条目来源。这种约束生成的方式是被认为能提升法律智能问答系统的可靠性^[10]。

1.3 本文研究内容

对于传统法律咨询工具语义理解能力较弱、通用大模型在法律专业领域存在事实幻觉及知识更新滞后等问题，本文是要设计并实现一个基于检索增强生成（RAG）技术的法律智能问答系统。通过加上权威法律条文向量库与互联网实时搜索能力，让回答既能参考已有法律依据，也能结合部分实时信息。

在法律知识库构建与预处理方面，本文主要研究如何针对法律条文的层级结构的文档切分方法。这种方式可以提升切分后的内容在法律语境下的语义保留效果，并减少检索过程中的语义碎片化，从而提升长文本语义的捕捉能力^[7]。利用向量数据库，来实现法律知识的向量化存储^[17]，为后续的检索提供数据。

在检索机制与生成逻辑方面，本文采用了本地法律知识库与互联网搜索结合的方式。本地向量数据库主要负责提供较稳定的法律内容，搜索引擎则补充部分实时信息，两者一起使用，可以在一定程度上缓解模型知识更新不及时的问题，也减少大模型的“知识截断”问题^[11]。与此同时，针对生成端所需的可解释性需求，

本文也是使用了提示工程（Prompt Engineering）的方法^[18]。在实际生成时，会通过提示词对模型回答进行一定限制。借鉴 ChatLaw 的思想^[12]，构建结构化的指令模板，实现基于证据的推理，旨在为公众提供更实用且专业的法律咨询。

1.4 论文组织结构

本论文遵循从理论探索、需求分析到系统实现及效果验证的技术路线，采用“总—分”式结构对研究成果进行组织。全文共分为七章，具体组织结构安排如下：

第 1 章 绪论：本章主要概述了课题的研究背景与现实意义，并通过对国内外研究现状的分析，指出现有研究在法律应用中所面临的局限性与挑战。最后，对论文的整体组织架构进行了总结，为后续各章节的深入展开做好铺垫。

第 2 章 相关理论基础：本章梳理了本研究所涉及的核心理论体系，重点涵盖大语言模型、提示词工程、数据预处理方法以及向量数据库架构。

第 3 章 系统需求分析：本章从系统的应用场景，以及功能与非功能两个方面来开展的需求分析。也构建包含了协同检索、知识库预处理、提示词工程等核心功能的架构，同时提出了系统在响应时延等方面的要求。

第 4 章 系统设计：本章结合法律咨询的的实际的应用场景，从多个模块方面开展设计。重点论述法律知识库管理、多源协同检索、提示词工程以及用户交互界面的核心逻辑；同时给出了系统所需的数据库表。

第 5 章 系统实现：本章对法律大模型问答系统的核心实现过程进行了阐述，通过“运行效果、核心代码、技术解析”的逻辑，展示了从用户认证到 RAG 检索生成，再到后台管理的全链路工程落地。

第 6 章 系统测试：本章主要是进行了本系统与原生大模型的对比评测，我们从回答专业性、法条引用准确性等方面分析了此系统的表现。同时，还有一些性能指标测试，评估了系统在不同负载下的响应效率与工程稳定性。

第 7 章 总结与展望：本章对全文的研究工作与成果进行了总结，客观指出了研究中存在的局限性，并提出了后续可能的改进方向与研究展望。

第 2 章 相关理论基础

2.1 大语言模型

2.1.1 大语言模型概述

大语言模型的发展主要依赖深度学习中的自然语言处理技术的突破,其核心是建立在由当时在 Google 工作的 Vaswani 等人于 2017 年提出的 Transformer 架构之上^[4]。Transformer 放弃了早期的循环神经网络 (RNN) 结构,它不再依赖按顺序逐步处理的方式。而全面采用注意力机制。

Transformer 中使用的注意力机制是用来计算输入中不同词之间的关系。在处理文本时,每个词都会和其他词进行一定程度的关联计算,从而帮助模型更好地理解上下文信息。在具体计算中,输入会被转换为查询 (Q)、键 (K) 和值 (V) 三个向量,然后通过它们之间的匹配关系来计算每个词的重要程度。其计算方式如公式 2-1 所示。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2-1)$$

其中, d_k 为键向量的维度,引入分母 $\sqrt{d_k}$ 旨在防止点积过大导致梯度消失,确保大规模参数训练的稳定性。近年来,随着算力与数据规模的爆发,涌现出一批性能卓越的主流大模型,其基本情况如表 2-1 所示。

表 2-1 主流大语言模型核心参数对比

模型系列	模型名称	参数规模	上下文长度
GPT-5 系列	GPT-5.4	未公开 (推测 T 级)	1,000K tokens
Qwen 系列	Qwen3.5-Plus	397B	1,000K tokens
DeepSeek 系列	DeepSeekV-3.2	671B	128K tokens
Gemini 系列	Gemini 3 Pro	未公开 (推测 T 级)	1,000K tokens
Claude 系列	Claude 4.6 Opus	未公开 (推测 T 级)	1,000K tokens

大语言模型通过在大量文本数据上的训练,可以对自然语言中的含义进行一定程度的理解,同时能够完成一些比较复杂的文本分析任务等,展现出了类人智能特征^[19]。传统方法通常依赖规则或者关键词匹配,在遇到表达比较复杂或者语

境变化较大的情况时，容易出现匹配不准确的问题。相比之下，大语言模型在处理文本时会结合上下文信息一起进行判断，因此在理解法律相关表达时，会更关注具体语境中的含义，从而在一定程度上提高问答结果与问题之间的对应关系。

2.1.2 Qwen 系列大语言模型

本文在生成模块中选用了阿里巴巴开源的 Qwen（通义千问）系列模型作为基础模型。Qwen 是属于 Decoder-only 的架构，并通过大规模多语言语料进行了预训练，具备较好的文本理解与生成能力。考虑到本研究我们主要是面向中文法律问答场景，Qwen 在中文语义理解方面的表现能够较好满足系统需求。

在模型结构方面，Qwen 引入了 RoPE（旋转位置编码）机制^[20]。与传统绝对位置编码方式相比，该机制在处理长文本时具有一定优势，能够较好适应法律条文这些篇幅较长的文本内容。此外，模型在预训练过程中积累了对多种文本形式的理解能力，使其在结合外部法律知识进行问答生成时，能够更好地理解法条内容与用户问题之间的关联。

2.2 检索增强生成的核心原理

2.2.1 RAG 技术概述

检索增强生成（Retrieval-Augmented Generation, RAG）是 Lewis 等人在 2020 年提出的一种结合信息检索与文本生成的方法^[6]。其主要思路是在大语言模型生成回答前，先从外部知识库中检索与问题相关的信息，再将检索结果作为上下文输入模型，来辅助后续内容生成。相较于仅依赖模型参数中已有知识的方式，RAG 能够在一定程度上缓解大语言模型因知识更新不及时或相关领域的专业的知识不足而导致的回答偏差问题。

2.2.2 RAG 的基本流程

检索增强生成是一种结合了非参数化知识（外部向量数据库）的技术范式。其核心流程分为三步：

（1）检索（Retrieval）阶段

系统接收到原始输入查询（Query）后，利用嵌入模型（Embedding Model）将其转化为高维向量。随后，在向量数据库中利用相似度算法（如余弦相似度）进行快速搜索，从预存储的非参数化知识库中召回的几个。

(2) 增强 (Augmentation) 阶段

完成相关内容检索后，系统会将检索结果与用户提出的问题进行组合，并按照提示词模板 (Prompt Template) 整理后输入给大语言模型。这样做的原因是，希望模型在回答问题时能够同时参考检索到的法律条文、案例或者其他相关内容，而不是只依赖模型本身已有的信息。尤其是在法律咨询场景下，有外部内容作为参考，也能够减少回答与实际法律依据出现偏差的情况。

(3) 生成 (Generation) 阶段

大语言模型作为生成器，对增强后的提示词进行深度解析。模型不再仅依赖其预训练阶段习得的内部参数化知识，而是优先参考上下文中的非参数化证据，通过内部逻辑推理链生成具备事实支撑的最终回答。

2.3 提示词工程

2.3.1 提示词工程概述

提示词工程 (Prompt Engineering) 是指通过设计输入内容和提示表达方式，来引导大语言模型完成特定任务的一个方法^[18]。在实际应用中，模型输出结果会受到问题描述方式的影响，这就需要通过合理组织提示内容，使模型能够更准确地理解任务需求。一般情况下，提示词中会包含一些任务说明、约束条件以及上下文信息，以帮助模型生成更符合要求的回答。对于用于回答用户问题的大模型而言，提示词设计会直接影响模型输出效果，所以在不同任务场景中，往往需要结合实际需求对提示内容进行调整与优化。其作用主要体现在以下两个方面：

- (1) 语义空间激活：利用特定的前缀指令 (Prefix Tuning) 和语境锚点，激活大模型在预训练阶段存储的相关领域知识，使其从通用回复模式切换至法律这个专业对话模式。
- (2) 约束与引导机制：通过预设角色设定 (System Prompt)、任务限制和格式规范，构建一个受控的生成环境。这种机制能显著压制模型的“随机性抽样”，确保输出结果在事实准确性与逻辑一致性上满足工程化应用的要求。

2.3.2 提示词设计原则与本系统的应用

因为法律问题对回答要求比较高，所以这里专门设计了一套适用于法律场景的提示词方式提示词模板^[11]。在实际流程中，系统会把用户的问题、检索到的法律条文、本地资料和部分网络信息整理到一起，然后一起交给模型处理。这样做

的目的是让模型在回答前能看到更多相关内容，而不是只根据单一问题生成回答。

在提示词内容上，主要会包含几个部分，比如当前的问题、相关背景信息，以及对回答的一些基本要求，这样模型生成的内容会更完整一些，也更容易贴合实际法律场景。同为了避免一些不相关的问题进入系统，前面还加了一层判断，用来识别问题是否属于法律相关内容，同时也会对问题做一定的扩展，让模型可以从不同角度理解同一个问题。

此外，在连续对话过程中，系统会结合历史对话信息显示的对话内容，这样模型在回答时可以知道上下文在说什么，避免出现前后不一致的情况。并参考思维链（Chain of Thought, CoT）技术^[21]，让模型在处理问题时更容易结合上下文来理解用户的真实意图。

最后，系统通过总结与解释性的提示方式，将较为专业的法律内容转化为相对容易理解的表达，在保证我们系统生成的法律内容准确性的同时也到兼顾普通用户的阅读需求^[22]。

2.4 向量表征与数据预处理技术

2.4.1 文本嵌入原理

文本嵌入（Embedding）是指将文本内容转换为向量形式。在法律检索场景中，Embedding 模型会对法律文本的语义信息进行转化，并将法条、案例或用户问题表示为固定长度的向量^{[11][13]}。

和传统的关键词匹配方式不同，这种方法不只是看字面是否一致，还会参考整体表达的相似程度。所以即使说法不一样，只要意思接近，也有可能被匹配到。

比如，“解除劳动合同”与“终止劳动关系”在字面上不同，但在法律语境中表达的内容比较接近，因此在检索时通常会被认为是相关内容。

2.4.2 相似度度量算法

在 RAG 架构中，检索阶段主要是用来判断用户问题和知识库中内容的相似程度。具体做法是，先把用户问题和文本片段都转换成向量，然后通过计算这些向量之间的相似程度来进行匹配。

本系统采用余弦相似度（Cosine Similarity）作为计算方式，它通过判断两个向量夹角的大小来衡量它们的接近程度。一般来说，数值越大，表示两段文本在含义上越接近，其计算公式如公式 2-2 所示：

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|} \quad (2-2)$$

通过这种方式，系统能够突破关键词匹配的限制，实现深层的语义对齐，从而在复杂的法律表述中精准定位相关依据。

2.4.3 法律长文本的递归切分策略

法律文档一般篇幅较长，而且结构比较清晰，比如会分成不同的章节和条款。如果直接整段输入模型，一方面会比较占用处理资源，另一方面也可能超过模型的长度限制，所以需要先把文本拆开处理，也就是对文本进行分块（Chunking）处理^[6]。

本研究在处理时采用递归字符切分的方法，把长文本拆成多个小片段。和按固定长度切分不同，这种方法会尽量先按照标题或者自然分隔位置来切分，然后再控制每一段的长度在合理范围内。

这样处理之后，每一段内容会相对完整一些，不容易出现上下文被截断的情况。比如在法律条文里，一条规则的条件和对应的处理结果通常是连在一起的，如果被拆开，就可能影响后续检索和理解。

2.5 系统架构技术

本系统在架构设计上采用了现代化的全栈技术栈，涵盖了前端交互、后端逻辑处理及数据库数据存储技术。

2.5.1 前端开发框架

本系统前端使用 Vue 3 和 Element Plus 来搭建页面。

Vue 3 主要用来处理页面结构和交互逻辑，开发时采用组合式 API 来组织代码，这样不同功能的逻辑可以分开写，后面修改起来也更清晰一些。同时它的响应式机制可以让页面数据变化时自动更新界面，不需要手动刷新。

在页面样式方面，使用了 Element Plus 组件库，里面已经提供了按钮、表格、弹窗和布局等常用组件，所以很多基础页面可以直接复用这些组件来搭建，整体风格也比较统一。

在实际交互中，系统支持问答内容的动态更新，用户可以在回答生成过程中看到内容逐步输出，而不是等全部生成完成后再显示。

2.5.2 后端开发框架

FastAPI 是基于 Python 的一个 Web 框架，主要用来处理前后端之间的数据请求。在本系统中，法律问答流程会涉及到模型调用和知识检索，这些步骤本身会有一定的等待时间，所以后端需要能同时处理多个请求。FastAPI 在处理这类并发请求时表现也比较灵活，当某个请求在等待模型返回结果时，系统可以继续处理其他请求，不会被卡住，从而让整体响应过程更顺畅一些。

2.5.3 向量数据库

针对大规模非结构化数据的语义检索需求，系统采用了 ChromaDB 作为向量数据库。ChromaDB 是一款专为 AI 原生应用设计的开源向量嵌入数据库，专注于高维向量的存储与快速最近邻搜索。是构建 RAG 系统不可或缺的底层基础设施。

2.5.4 关系型数据库

系统在结构化数据管理上使用 SQLite 来存储数据。SQLite 本身是一个比较轻量的数据库，数据直接保存在本地文件里，不需要单独搭建数据库服务。在这个系统里，用户信息、历史对话以及收藏记录这些结构化数据都放在 SQLite 中进行管理。因为整个系统是偏应用型的开发，不需要复杂的数据库集群，所以用 SQLite 可以减少很多配置步骤，开发和测试时也会更方便一些。

2.6 本章小结

本章主要对系统实现过程中涉及的相关理论与技术进行了介绍。首先，我们对大语言模型及其基础架构 Transformer 进行了说明，并结合系统实际需求，介绍了所采用的 Qwen 系列模型及其在中文法律问答场景中的适用性。随后，对 RAG 相关内容进行了分析，包括系统中知识检索与回答生成之间的基本流程。又结合了本研究的法律咨询场景，对提示词工程的设计思路进行了说明，同时介绍了文本向量化、余弦相似度计算以及递归字符切分等处理方法。最后，对系统前后端所采用的技术框架进行了简要介绍，包括 Vue 3、FastAPI、ChromaDB 和 SQLite 等内容，为后续系统设计与功能实现提供了相应的技术准备。

第3章 系统需求分析

3.1 需求概述

本系统的主要使用对象为缺乏法律专业知识的普通用户，同时也可作为初级法律从业者提供一定参考。在实际使用过程中，系统主要面向以下几类场景：

- (1) 日常法律咨询：当用户在劳动纠纷等问题中遇到法律疑问时，可以通过大白话描述具体情况。系统会结合检索到的法律条文及相关资料，对问题进行分析，并给出相应的法律依据与参考建议。
- (2) 法律知识检索：用户可以输入法律术语或以较为模糊的问题形式进行提问。系统会结合本地法律知识库和互联网信息，对法律内容进行查询，并返回对应法条或解释性内容，帮助用户了解相关法律规定。

3.2 业务功能分解

本系统主要围绕法律咨询场景进行设计，其整体功能结构如图 3-1 所示。在用户交互这一个方面，用户可以通过大白话来描述法律问题，系统结合检索到的法律知识内容对应回答，以提高回答与问题之间的相关性。同时，系统还提供会话管理功能，如支持历史咨询记录查看等操作。在知识检索部分，系统结合本地法律知识库与互联网信息，来兼顾法律条文内容与部分实时信息需求。同时系统要能完成文本切分以及向量化存储等工作，为后续法律问答提供数据支持。

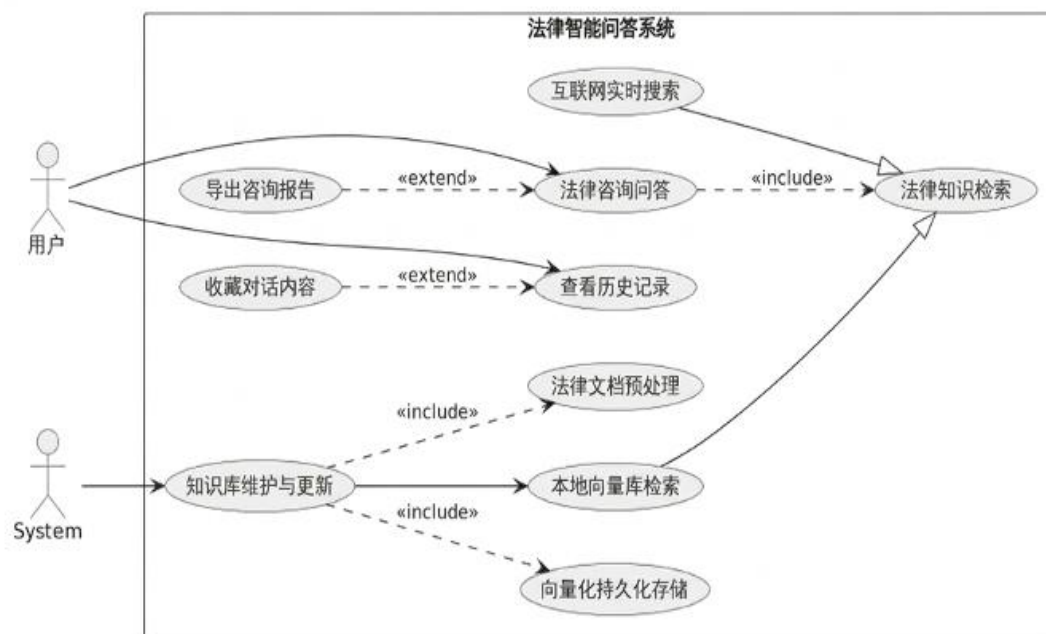


图 3-1 系统用例图

3.3 功能性需求分析

结合前文所述的应用场景以及 RAG 架构的实现方式，本系统对法律咨询流程进行了功能设计，整体包括数据处理、知识检索和回答生成等环节。根据系统实际使用需求，主要可划分为智能问答、知识库管理以及会话管理三个方面。

3.3.1 智能法律问答功能需求

作为系统的核心业务逻辑，需要负责处理用户意图并调度异构知识源，其具体子任务如下：

- (1) 意图识别与过滤：准确识别用户输入是否属于法律咨询对问题。通过提前准备好的校验逻辑，来过滤与法律无关的问题。
- (2) 指代消解与重写：在多轮对话过程中，用户可能会使用“它”“这种情况”“这样做”等指代表达，导致问题含义不够明确。为次，系统会结合前文对话内容，对用户问题进行补充和调整，使其能够形成语义更加完整的查询内容，从而帮助后续检索更准确地理解用户意图。
- (3) 多源检索融合：系统需支持并行检索任务，即同时调用本地向量数据库（获取权威法条）与互联网搜索引擎（获取最新资讯），实现法律确定性与时效性的互补。

其中核心部分的多源检索融合的用例分析如表 3-1 所示：

表 3-1 多源检索融合用例分析表

用例编号	UC-001
用例名称	多源检索融合
参与者	用户、系统后台
优先级	高
用例描述	用户提问后，系统并行调度本地向量库与互联网搜索，获取法律数据。
触发事件	提问重构完成后触发检索任务
前置条件	数据库连接正常，模型调用 API 有效
基本事件流	1) 系统将重构后查询语句转化为向量表示。 2) 并行执行本地 ChromaDB 向量库检索。 3) 同步执行互联网实时搜索（获取最新案例）。

	4) 将结果聚合通过回答展示给用户。
异常事件流	1) 若网络连接异常则仅展示本地法条结果。 2) 若 API 调用异常，则在后台提示错误。
后置条件	系统获得多源协同的法律数据

3.3.2 知识库管理功能需求

系统需要能够进行法律知识的非参数化存储与预处理, 这也是系统专业性的基石, 主要分为如下两个操作:

- (1) 法律文档预处理: 支持 Markdown 格式的法律文书导入。系统需实现递归式语义切分, 根据法律文本的层级逻辑 (如章、节、条) 自动识别断点, 确保数据切片的语义完整性。
- (2) 向量化存储: 将切分后的文本映射为高维向量, 并持久化存储于 Chroma 向量数据库中。同时, 系统需支持对切片关联元数据 (如法律名称、发布日期) 的标记, 以便进行多维度的混合检索。

其用例分析表如 表 3-2 所示:

表 3-2 知识库管理与维护用例分析表

用例编号	UC-002
用例名称	知识库管理与维护
参与者	系统后台
优先级	高
用例描述	系统对导入的 Markdown 法律文档进行语义切分及向量化处理, 并持久化存入向量数据库中, 作为非参数化知识源。
触发事件	发出系统数据同步更新指令
前置条件	法律文档符合 Markdown 格式规范; Chroma 数据库服务运行正常。
基本事件流	1) 系统读取原始原始法律文本。 2) 执行递归式语义切分: 识别“章、节、条”层级逻辑, 自动定位断点。 3) 调用模型将文本转化为高维向量。

	4) 将向量数据与关联元数据同步持久化存储至 Chroma 向量数据库中
异常事件流	1) 若文档格式非法或层级混乱，中止该文件处理。 2) 若向量化模型连接超时，系统开启重试机制。
后置条件	系统完成非参数化知识的更新，向量数据库具备可供混合检索的结构化向量索引。

3.3.3 会话管理功能需求

为了提升用户的使用体验并确保法律咨询过程的连续性，系统需具备完善的会话持久化与数据交互功能如表 3-3 所示，功能主要分为如下三点：

- (1) 历史记录持久化与回溯：系统需要对用户的咨询内容进行保存，并按照会话进行管理。用户也可以查看历史对话内容，方便回顾之前的咨询过程。
- (2) 对话收藏功能：对于用户认为较为重要的咨询内容，系统支持收藏操作。用户可以将需要保留的对话加入收藏列表，后续查看时能够更方便地找到相关内容，来再次进行查看。
- (3) 咨询结果导出：系统需支持将完整的法律咨询内容导出为标准的 PDF 文档。导出的文档应包含用户提问以及系统生成的专业意见，方便用户保存查看。

表 3-3 会话管理与数据交互用例分析表

用例编号	UC-003
用例名称	会话管理与数据交互
参与者	用户
优先级	中
用例描述	系统实时持久化存储咨询历程，支持用户对历史会话的回溯、重要对话的收藏以及咨询结果的标准化 PDF 导出。
触发事件	用户点击历史列表、收藏按钮或导出指令
前置条件	用户处于登录状态且当前或往期存在有效的对话记录。

基本事件流	1) 系统实时记录咨询提问与生成结果，并关联会话 ID 持久化至数据库。 2) 用户调取历史列表，系统根据会话 ID 还原往期对话上下文以供回溯追问。 3) 用户点击收藏按钮，系统将指定对话片段标记并同步至用户收藏夹。 4) 用户发起导出请求，系统提取当前会话的提问与专业建议，渲染并生成标准 PDF 文档供用户下载。
异常事件流	1) 若数据库连接中断，系统提示无法加载历史记录。 2) 若 PDF 渲染引擎报错，系统提示重试。
后置条件	系统完成数据状态的同步更新（如收藏标记），并输出可持久保存的法律咨询结果文档。

3.4 非功能性需求

3.4.1 准确性与权威性要求

由于法律咨询对内容准确性要求较高，系统在回答生成过程中需要尽量保证输出内容与法律依据保持一致。在回答生成阶段，系统主要按照“先检索、后生成”的方式进行处理，通过提示词约束模型优先参考本地知识库中的法律条文与相关内容，减少因模型自身生成带来的错误信息，并避免出现不存在的法律条文或表述不准确的情况。

此外，为了方便用户对咨询结果进行核对，系统还需要提供一定的可追溯能力。生成回答时，会尽量附带相关法条信息或来源链接，使用户能够进一步查看原始法律依据，并结合实际情况进行判断，以提高法律咨询结果的参考价值。

3.4.2 性能性要求

由于 RAG 问答过程需要经过知识检索、网络信息获取以及大模型生成等多个步骤，系统在回答问题时可能存在一定处理耗时。因此，在性能设计方面，需要尽量减少用户等待时间。本系统结合 FastAPI 的异步处理方式以及流式输出技术（Server-Sent Events, SSE），使回答内容能够逐步返回，用户无需等待完整结果生成后再查看，从而改善交互体验。此外，考虑到系统需要在较多法律文本中完成相关内容检索，还需要兼顾检索效率与系统响应速度。在网络环境正

常的时候，系统应尽量保证咨询结果能够在较短时间内返回，以满足日常法律咨询场景的使用需求。

3.4.3 可靠性与稳定性要求

系统可靠性主要体现在数据保存以及知识库运行稳定性两个方面。在数据管理方面，系统采用 SQLite 对用户信息、历史会话以及收藏记录等内容进行本地存储，以保证数据能够被持续保存。考虑到程序运行过程中可能出现异常、网络中断之类情况，系统需要尽量减少用户历史数据丢失的风险，保证后续仍能够正常查看已有记录。在知识库维护方面，系统支持法律文档的新增与更新处理。当进行文档导入和向量化操作时，应尽量保证知识检索功能能够正常运行，以满足系统持续使用的需求。

3.5 本章小结

本章主要围绕法律咨询系统的需求进行了分析，并结合 RAG 架构的实现方式，对系统在实际使用中的功能进行了梳理。通过前文分析，我们明确了系统的主要应用场景，主要是日常法律咨询等内容。在功能设计方面，本研究对智能问答、会话管理以及知识库维护等需求进行了说明，同时也考虑了多轮对话中的问题补充、历史记录查看、对话收藏和 PDF 导出等功能，以满足用户在法律咨询过程中的实际使用需求。此外，我们还从系统使用角度出发，对回答准确性、响应效率以及数据稳定性等方面进行了简单说明，希望系统在保证法律内容参考价值的同时，也能够具有较好的使用体验。本章内容也为后续系统设计与具体功能实现提供了基础。

第 4 章 系统设计

4.1 系统总体架构设计

在系统总体功能设计中，本系统结合法律咨询场景的实际需求，对整体功能进行了模块划分。如图 4-1 所示，系统整体主要由会话管理与交互模块、智能法律问答模块、知识库预处理模块以及后台与系统管理模块组成。各模块之间相互配合，共同完成从法律数据处理到咨询结果生成的一个整体流程。

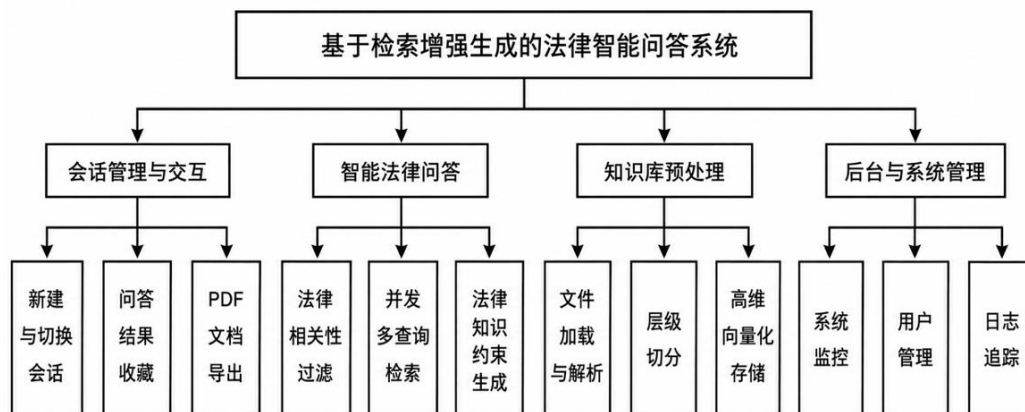


图 4-1 系统总体框架图

会话管理与交互模块主要负责用户与系统之间的交互过程。它支持新建会话、切换历史会话等操作，用于管理不同的法律咨询内容的记录。同时，系统还提供对话收藏和 PDF 导出功能，方便用户来进行保存重要的内容。

智能法律问答模块是系统实现法律咨询功能的核心部分。这个模块结合提示词设计以及知识检索结果，对用户提出的问题进行分析与回答。在回答生成前，系统会先进行法律相关性判断，并结合多条检索结果提高回答内容与问题之间的匹配程度，来尽量减少错误的回答，提高法律咨询结果的参考价值。

知识库预处理模块主要用于法律知识内容的整理与存储。系统会对原始法律文档进行读取、解析和文本切分，再通过向量化方式完成存储，为后续法律检索提供支持。

后台与系统管理模块主要负责系统运行过程中的基础管理工作，包括用户管理和部分系统状态维护等内容，以保证系统能够较稳定地运行。

4.2 核心业务流程设计

本系统的核心业务流程旨在构建一个从用户自然语言输入到专业法律建议输出的“检索增强生成 (RAG)”流水线。系统通过问题校验、多路径检索增强以及上下文融合生成，确保回答的专业性与时效性。核心业务流程如图 4-2 所示。

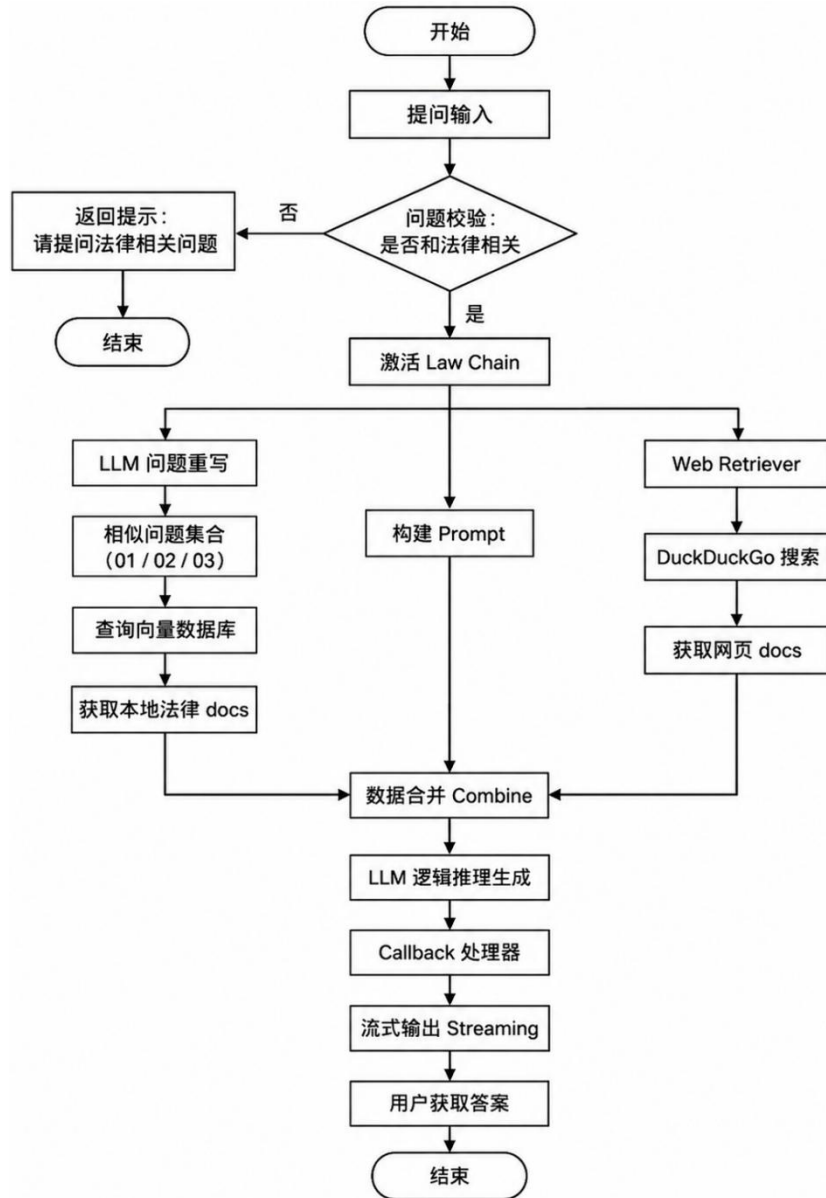


图 4-2 核心业务流程图

(1) 提问校验

当用户在前端页面提交法律咨询问题后，系统会先进入问题的校验环节，其处理流程如图 4-3 所示。该阶段主要用于判断用户输入内容是否与法律问题相关。系统会结合大语言模型对问题进行初步分析，判断其是否属于法律咨询场景。

如果识别结果显示问题与法律无关，系统会提示用户输入法律相关内容，并结束当前问答流程；如果判断为法律问题，则继续进入后续处理阶段，对问题进行知识检索与回答生成。通过增加这一判断步骤，可以在一定程度上减少无关问题对系统处理过程的影响，使系统更聚焦于法律咨询场景。

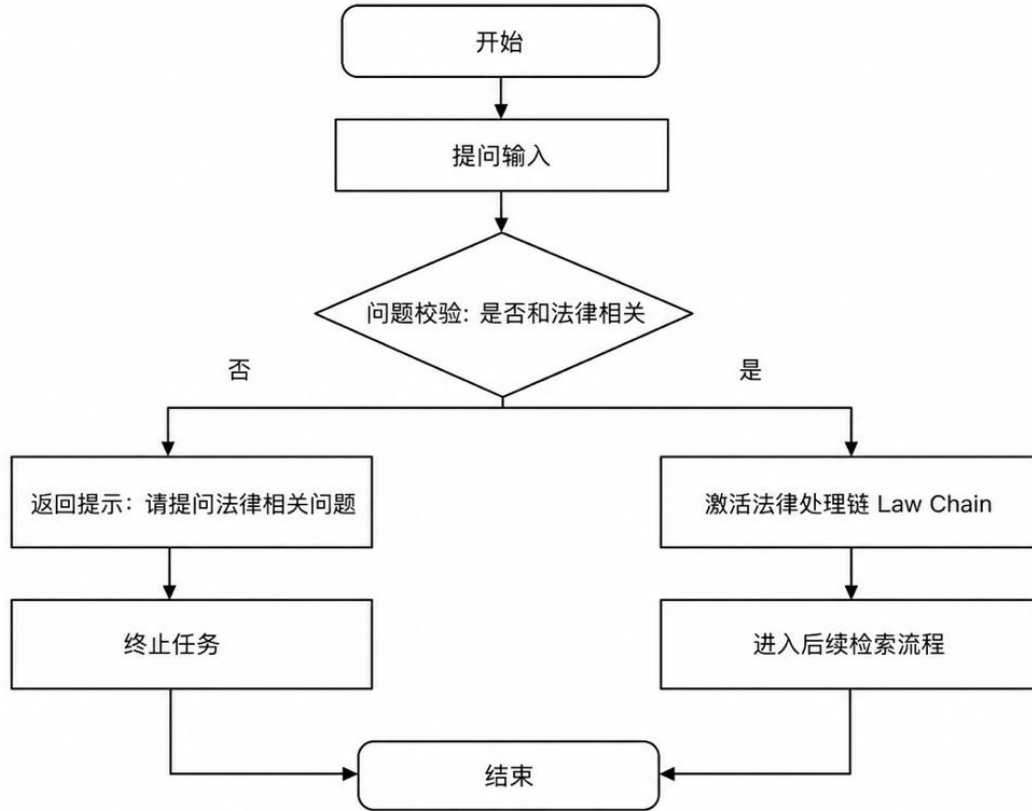


图 4-3 提问校验流程图

(2) 并行多源检索增强

在法律处理链内部，系统采用并行架构调度两类异构检索模块以扩充知识背景，局部逻辑如图 4-4 所示。

- 1) MultiQuery Retriever（多查询检索）：针对法律术语表达的多样性，系统调用大语言模型对原始问题进行重写，生成 3 个语义相近的衍生问题。这些问题进入 Chroma 向量数据库进行相似度检索，通过多维度的查询提升了本地法律文档（法律 docs）的召回率。
- 2) Web Retriever（互联网检索）：系统同步调用 DuckDuckGo 搜索引擎执行全网检索，获取最新的司法解释及社会热点案例。该并行机制实现了本地权威法条与互联网时效信息的深度互补。

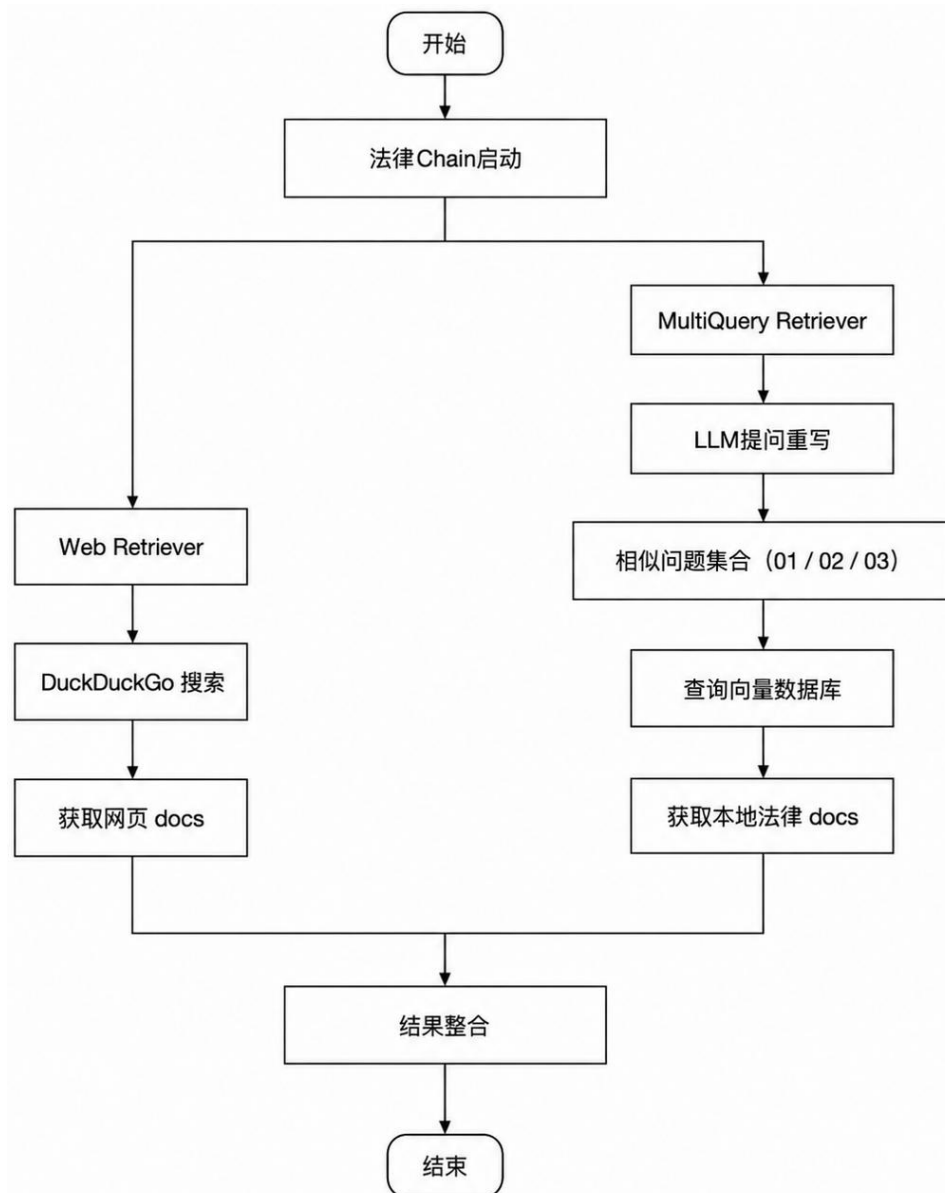


图 4-4 并行多源检索增强流程图

(3) 上下文融合与流式输出

在回答生成阶段，系统会进入知识整理与结果返回环节，处理过程如图 4-5 所示。系统先对多查询检索得到的本地法律文本、本地知识库中的相关内容以及互联网检索结果进行整理，再与提前设定好的提示词一起交给大模型处理。这样做的目的是让模型在回答时能够同时参考用户问题以及相关法律内容，而不是只依赖模型本身已有的信息。随后，整理后的内容会被输入到大语言模型中生成回答。为减少用户等待时的停顿感，系统采用流式输出方式，将回答内容逐步返回到前端页面，用户无需等待完整结果生成后再统一显示，而是可以边生成边查看。

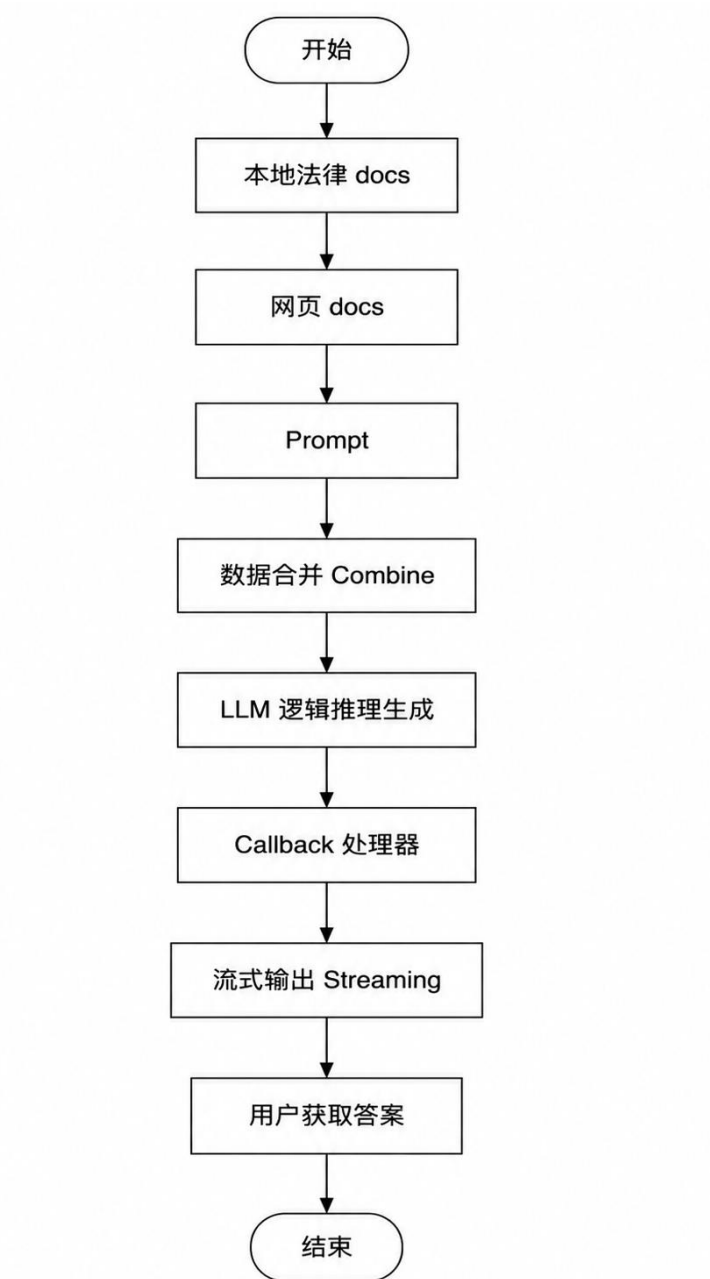


图 4-5 上下文融合与流式输出流程图

为了进一步阐明系统各组件间的动态交互逻辑,图 4-6 展示了用户发起咨询到获取答案的完整时序关系。

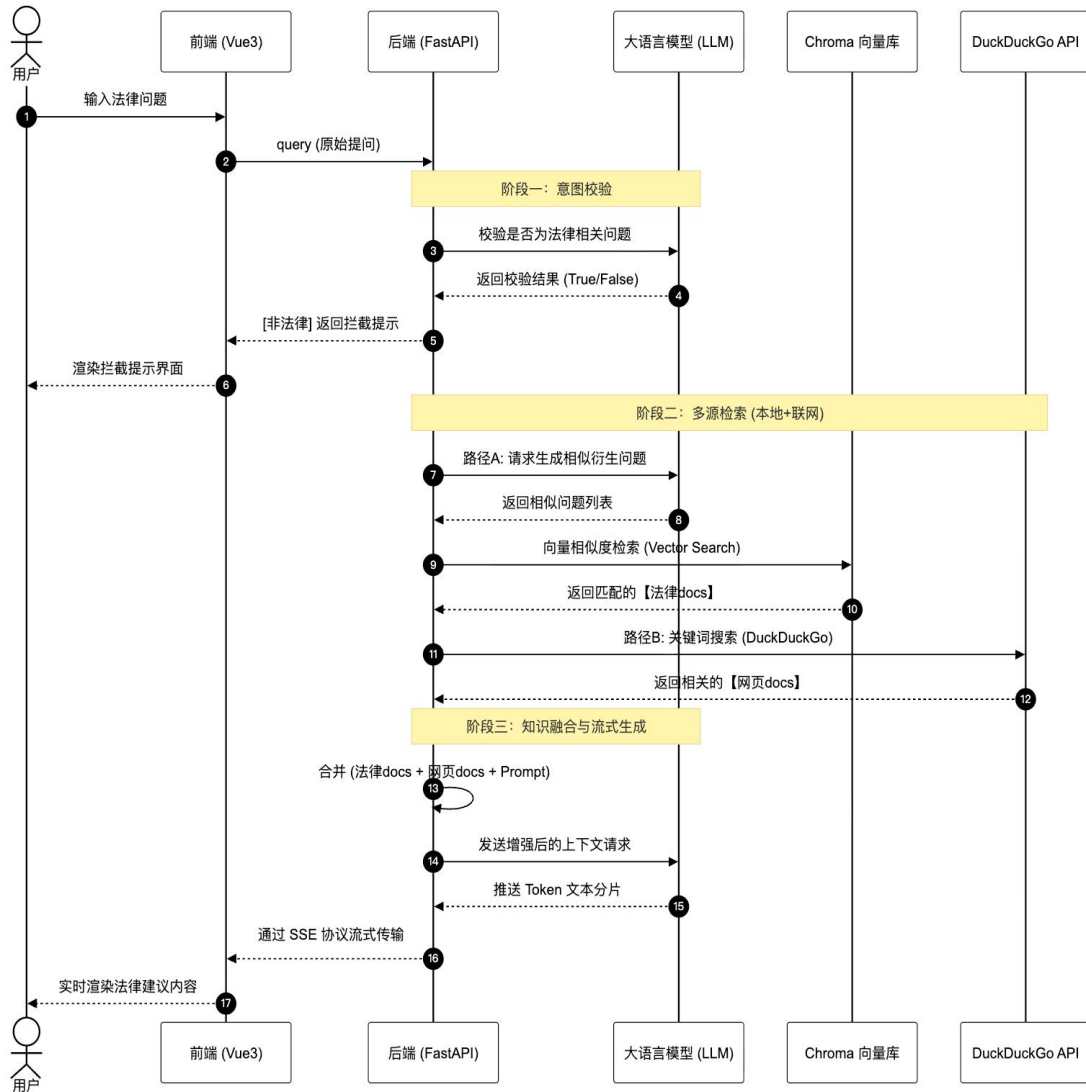


图 4-6 系统核心业务时序图

4.3 功能模块详细设计

4.3.1 知识库管理模块设计

知识库作为法律 RAG 系统的专业基石，负责将非结构化的法律文书转化为机器可理解的向量表示。本模块通过构建自动化的数据处理流水线，实现了从原始文档到高维向量索引的深度转化，其处理流程的活动图如图 4-7 所示。

(1) 法律文件加载与层级解析

系统首先通过 LawLoader 模块对原始 Markdown 格式的法律文件进行加载。为了保留法律条文天然的层级结构（如章、节、条），系统在 LawSplitter 单元中采用了两级切分机制。第一阶段利用 MarkdownHeaderTextSplitter 识别文档中的层级标题（如“#”与“##”），将文档初步划分为具有逻辑关联的结构化块。

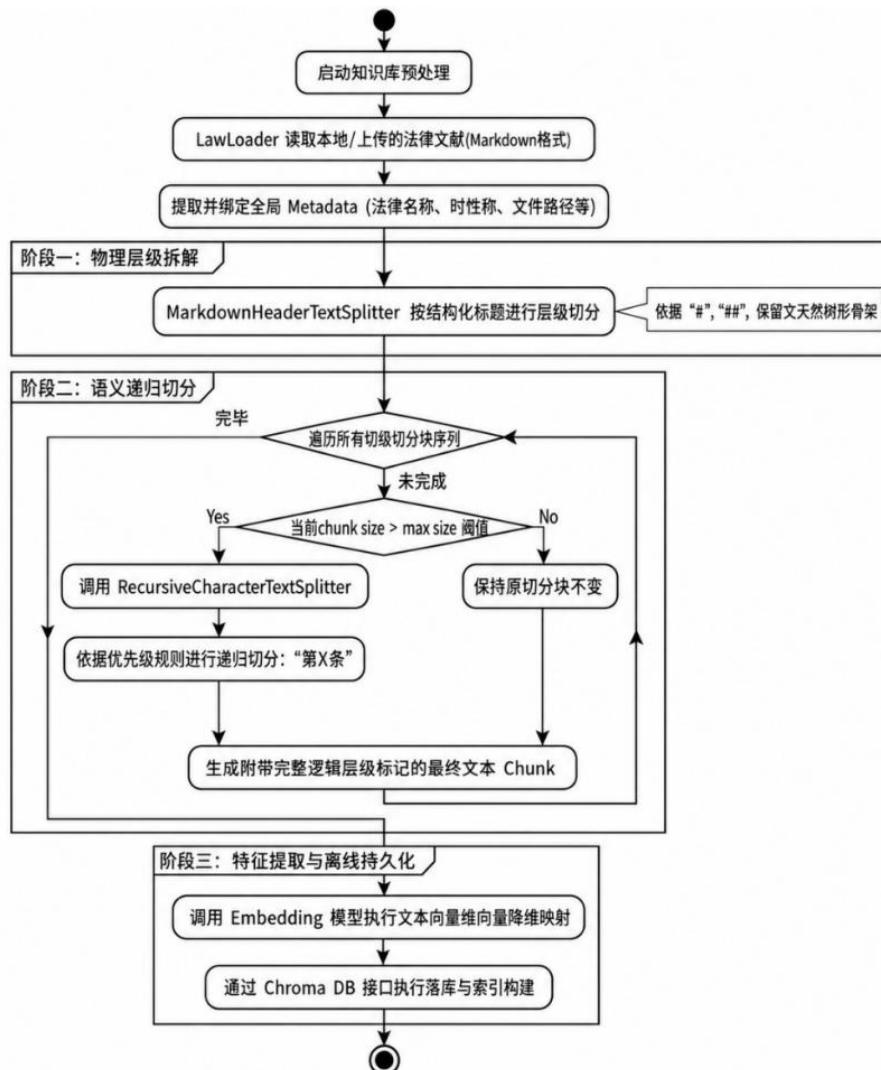


图 4-7 知识库预处理与存储活动图

(1) 递归式语义切分策略

在完成初步文本解析后，系统进一步使用 `RecursiveCharacterTextSplitter` 对法律文本来进行切分处理。考虑到法律文档通常具有较强的层级结构，系统会优先按照“#### 条”等标题标识以及部分标点符号进行划分，同时结合设定的文本长度限制，对内容进行递归的切分。通过这种方式，可以在控制文本长度的同时，尽量保留法律条文的完整内容，减少因切分位置不合理而导致的信息缺失。

(2) 向量化表示与持久化存储

切分后的文本块经 `Embedding` 嵌入模型处理，被映射为高维向量。随后，这些向量数据连同法律名称、发布日期等关键元数据 (Metadata) 一并存入 `Chroma` 向量数据库中。让系统能够实现语义相似度检索，并支持基于元数据的处理，从而满足法律业务对检索的需求。

4.3.2 智能法律问答模块设计

本系统的法律问答过程主要依靠提示词来进行引导。考虑到法律咨询场景对回答准确性和逻辑性的要求较高，系统针对不同处理阶段设计了相应的提示内容，包括问题校验、问题改写、知识检索以及回答生成等部分，以帮助模型更好地理解用户问题并结合相关法律信息进行回答。其整体处理流程如图 4-8 所示：

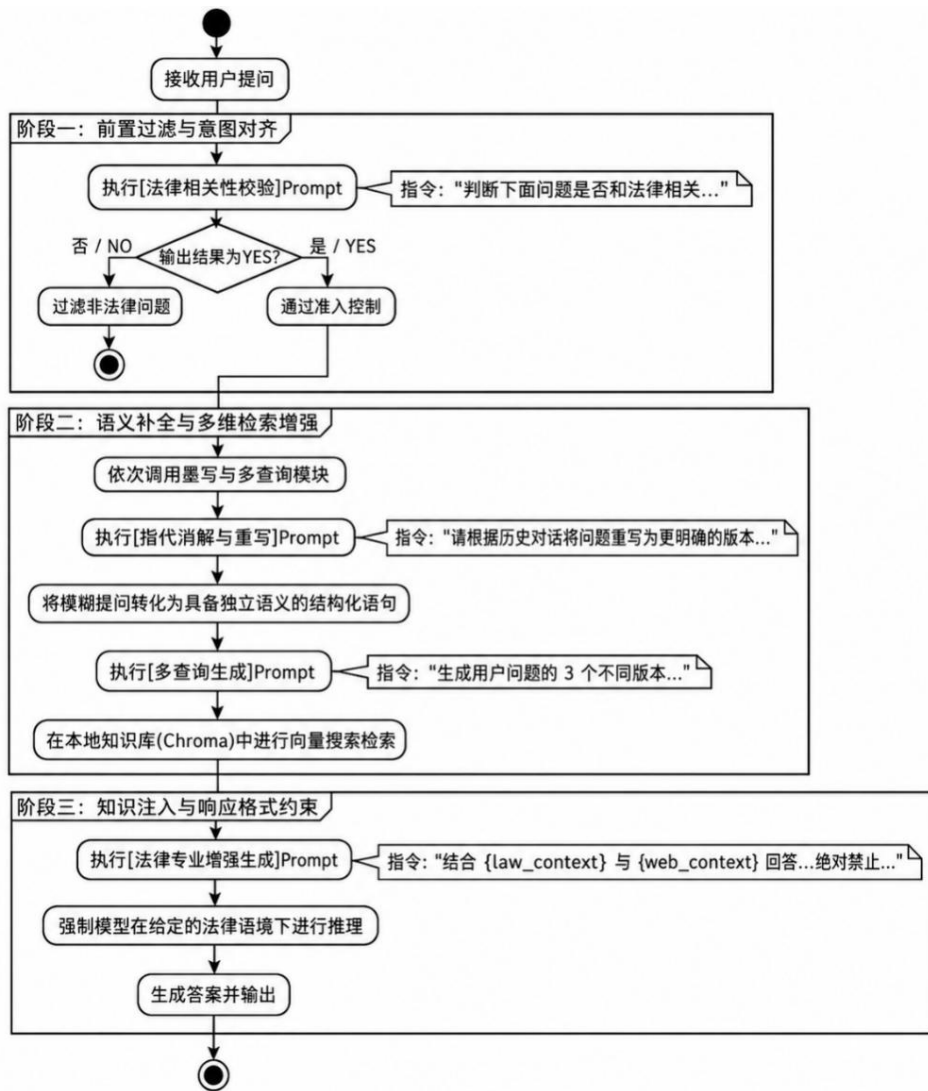


图 4-8 智能法律问答处理逻辑活动图

在具体问答流程开始之前，系统会先根据不同任务调用对应的提示词模板（Prompt Template）。为了便于后续使用与管理，系统将提示内容按照固定格式进行组织，并根据不同处理阶段填入相应信息。

在运行过程中，系统会结合当前任务自动选择对应模板，并也补充用户问题（{question}）、历史对话内容（{history}）、检索得到的法律文本（{law_context}）

以及外部搜索结果（{web_context}）等信息，使模型能够结合上下文完成后续处理。通过这种“提示模板 + 上下文信息”的方式，系统能根据不同场景调整提示的内容，减少重复编写提示词的情况，同时也方便后续功能扩展与维护。

为了来适应法律问答过程中不同阶段的处理需求，系统设计了多种提示词模板，分别用于法律相关性判断、问题改写、多查询生成以及回答生成等任务。同时，为了让回答的内容更容易被人理解，就对部分输出结果进行相应的表达调整。各提示词模板在运行过程中所使用的输入变量，及其基本结构如表 4-1 所示。

表 4-1 系统全局核心提示词模板定义表

阶段与模板标识名	动态输入变量	提示词核心模板（节选）
意图识别阶段	{history}	你是一个专业律师，请判断下面问题是否和法律相关，相关请回答 YES，不相关请回答 NO...
CHECK_LAW_PROMPT	{question}	历史对话: {history} 当前问题: {question} 判断规则: 如果问题直接提到法律、法规等概念回答 YES；如果是日常生活问题直接回答 NO...
语义补全阶段	{history}	你是一个专业的律师助手。请根据历史对话将问题重写为更明确的版本。 历史对话: {history} 当前问题: {question} 请直接输出重写后的问题，不要添加任何解释...
REWRITE_QUESTION_PROMPT	{question}	
检索发散阶段	{question}	您是 AI 语言模型助手。您的任务是生成给定用户问题的 3 个不同版本，以从矢量数据库中检索相关文档。通过对用户问题生成多个视角，帮助克服相似性搜索的限制。问题: {question} 提供这些用换行符分隔的替代问题...
MULTI_QUERY_PROMPT_TEMPLATE		
知识生成阶段	{law_context}	你是一个专业的律师，请你结合以下内容回答问题。 {law_context} {web_context} 历史对话: {history} 当前问题: {question} 回答要求: 绝对禁止使用任何 HTML 标签...
LAW_PROMPT_WITH_HISTORY	{web_context}	
	{question}	

交互优化阶段	{law_context}	你是一个专业且善于沟通的律师。请用通俗易懂
SUMMARY_PROMPT	{web_context}	懂的语言为用户总结回答。 法律依据:
	{question}	{law_context} 网络资料: {web_context} 用户
		问题: {question} 请注意: 用简单明了的语言
		解释, 避免过多术语...长度控制在 200 字以
		内...

从系统实现过程来看, 法律问答并不是直接依赖大语言模型一次性生成结果, 而是按照多个步骤逐步完成。为了提高回答过程的稳定性, 系统将法律问答拆分为问题校验、问题改写、知识检索、回答生成以及结果整理等多个阶段, 并通过不同提示词模板组织整个处理流程。

为了更清晰地描述这一过程, 将系统处理逻辑简化表示为多个阶段函数的组合, 其形式如公式 4-1 所示:

$$y = f_{summary} \circ f_{generate} \circ f_{retrieve} \circ f_{rewrite} \circ f_{check}(x) \quad (4-1)$$

其中, 输入 x 表示用户原始问题, 输出 y 表示最终返回结果各个函数如下:

f_{check} : 用于判断问题是否与法律咨询相关;

$f_{rewrite}$: 对用户问题进行补充或改写, 以减少多轮对话中信息不完整的情况;

$f_{retrieve}$: 结合向量检索与外部搜索获取相关法律内容;

$f_{generate}$: 结合检索到的信息生成对应回答;

$f_{summary}$: 对结果进行整理, 使回答更容易理解。

在检索的时候, 系统采用“多查询生成”这一策略, 通过提示词引导大模型生成了多个不同方面的表达 $Q=\{q_1, q_2, q_3, \dots, q_n\}$ 并分别执行向量检索:

$$D = \bigcup_{q_i \in Q} \text{Retrieve}(q_i) \quad (4-2)$$

在生成阶段, 系统基于检索增强生成 (Retrieval-Augmented Generation, RAG) 框架, 将外部知识作为条件输入注入模型, 如公式 4-3 所示:

$$y = \text{LLM}(x, D, H) \quad (4-3)$$

其中，D 表示检索得到的法律知识内容，H 表示历史对话信息。与仅依赖模型已有知识进行回答相比，该方式会结合外部法律资料进行辅助生成，从而减少回答内容与法律依据不一致的情况，也方便用户后续查看相关法律来源。

此外，为了使输出结果更加规范，系统还会在提示词中加入一定限制条件，例如回答格式要求、禁止输出 HTML 标签等，使生成内容更符合前端展示与法律咨询场景的使用需求。

结合前文的流程设计，系统的法律问答过程可以进一步划分为几个相互衔接的处理阶段。不同阶段对应不同任务，并由相应提示词模板进行处理，以完成从问题输入到结果输出的整体流程。

具体来说，系统主要包括以下三个阶段

(1) 前置过滤与意图对齐

系统在接收到用户问题后，会先进行法律相关性判断，其提示词设计如表 4-2 所示。该阶段主要用于识别用户输入是否属于法律咨询内容，来避免明显无关的问题进入后续处理流程。这个过程对应的是函数 f_{check} 。

具体来说，系统会结合提示词对用户问题进行分析，并要求模型仅返回“YES”或“NO”两种结果。当判断结果为“YES”时，系统继续执行后续知识检索与回答生成；若结果为“NO”，则提示用户输入法律相关问题，并结束当前流程。

该阶段的提示词设计通过引入明确的判定规则（如是否涉及法律术语、是否属于日常问题等），实现了对非法律问题的过滤，避免后面的资源回答浪费，提升了整体系统回答的效率。

表 4-2 法律相关性校验提示词设计

提示词名称	核心指令片段	设计意图与逻辑说明
法律相关性校验	"判断下面问题是否和法律相关..."	准入控制：通过严格的 YES/NO 输出约束，过滤非法律噪音。

(2) 语义补全与多维检索增强

在完成输入合法性校验后，系统进入问题补充与知识检索阶段，对应的函数是 f_{rewrite} 与 f_{retrieve} 。该阶段主要用于解决多轮对话中问题描述不完整，以及用户表达方式与法律术语存在差异等情况。

在此基础上，系统进一步对用户问题进行扩展处理。系统会基于提示词生成多个表达方式不同但含义相近的问题，然后分别执行知识检索，再对检索结果进行整理与合并。这样可以减少单一表述方式可能带来的检索的一些偏差，也可以帮助到来获取更多的相关的法律内容。

如表 4-3 所示，该阶段主要包括问题重写与多查询生成两类提示词设计。其中，问题重写主要用于补充上下文信息，提高问题表达的完整性；多查询生成则通过不同表达方式扩展检索范围，使系统能够覆盖更多可能相关的法律知识内容。

表 4-3 提问重写与检索增强提示词设计

提示词名称	核心指令片段	设计意图与逻辑说明
指代消解与重写	"请根据历史对话将问题重写为更明确的版本...直接输出重写后的问题。"	语义补全：将口语化提问转化为具备独立检索价值的语句，是提升向量空间检索精度的核心预处理步骤。
多查询生成	"生成用户问题的 3 个不同版本...帮助用户克服基于距离相似性搜索的限制。"	检索增强：利用模型发散思维生成视角变体，提高在 Chroma 中的召回率，有效弥补用户语言与法律术语间的语义差别。

(3) 知识注入与响应格式约束

在完成知识检索后，系统进入回答生成阶段，对应前文中的两个函数 f_{generate} 与 f_{summary} 。该阶段主要根据检索到的法律内容，对用户问题生成相应回答，并对结果进行整理，使其能够更容易理解。

具体来说，系统会将检索得到的法律文本（ $\{\text{law_context}\}$ ）以及外部补充信息（ $\{\text{web_context}\}$ ）一同输入模型，并结合用户问题进行回答生成。与仅依赖模型已有知识进行回答相比，这种方式能够结合具体法律内容进行辅助分析，从而减少回答内容与法律依据不一致的情况。

此外，为了使回答结果能够更稳定地展示在前端页面，系统还会在提示词中加入一定的格式要求，例如限制部分不必要的输出内容，避免生成影响页面展示的标签信息等。

考虑到普通用户可能较难理解专业法律表述，系统还增加了结果整理环节。在回答生成后，系统会对内容进行适当概括，并尽量使用较为通俗的语言重新表达，使用户能够更容易理解法律建议内容。

如表 4-4 所示，该阶段主要包括法律回答生成与结果总结两类提示词设计。其中，前面这个主要用于结合检索内容完成法律回答，后面这个则用于对结果进行整理与简化说明，来提高回答内容的可读性。

表 4-4 法律建议生成与总结提示词设计

提示词名称	核心指令片段	设计意图与逻辑说明
法律专业增强生成	" 结 合 {law_context} 与 {web_context} 回 答 ... 绝 对 禁 止 使 用 任 何 HTML 标 签。 "	知识注入：强制基于非参数化知识库推理。通过语法硬约束确保流式数据在前端界面的高可靠渲染。
通俗化总结	"用通俗易懂的语言总结回答...避免过多术语，长度控制在 200 字以内。 "	交互优化：提供“白话版”总结，降低法律专业门槛，满足用户对复杂法律咨询结果快速理解的需求。

4.3.3 会话管理与交互模块设计

为了提升用户的使用体验并确保法律咨询过程的连续性，系统构建了完善的会话持久化体系与轻量化交互机制。本模块主要负责历史记录的结构化存储、多轮对话的状态维护以及咨询结果的资产化导出。

(1) 会话持久化与状态管理

系统采用“会话 - 消息”的存储模型，通过 FastAPI 后端与关系型数据库协同实现。当用户进入系统时，可新建会话或在侧边栏切换历史会话。如流程的活动图 4-9 所示，用户的会话操作通过前端界面触发，通过认证中间件后与后端 SQLite 关系型数据库进行交互，保障所有用户的历史记录实现严格的私密隔离：

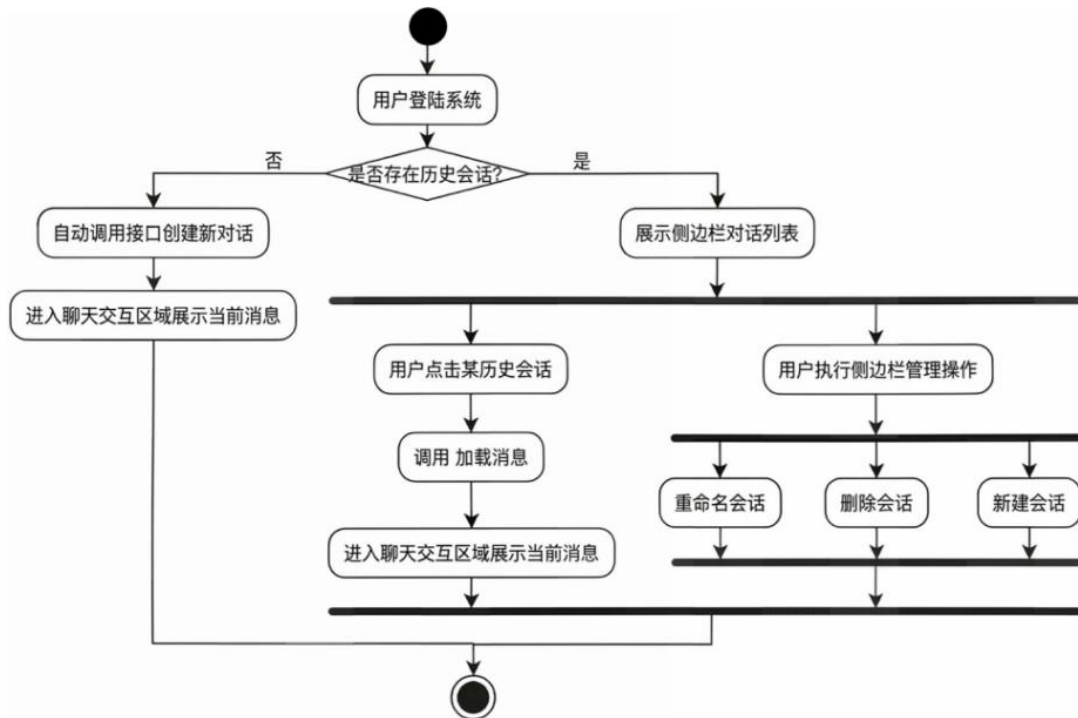


图 4-9 会话管理流程活动图

(2) 会话导出设计

对于法律类应用而言，将对话记录导出用以备忘或是作为法律意见是一项高频刚需。系统采用纯前端、无头导出的高效交互方案。在前端页面中，通过侧边栏会话的下拉菜单配置了触发项。如图 4-10 所示，PDF 导出的过程全部下放到用户浏览器中执行。

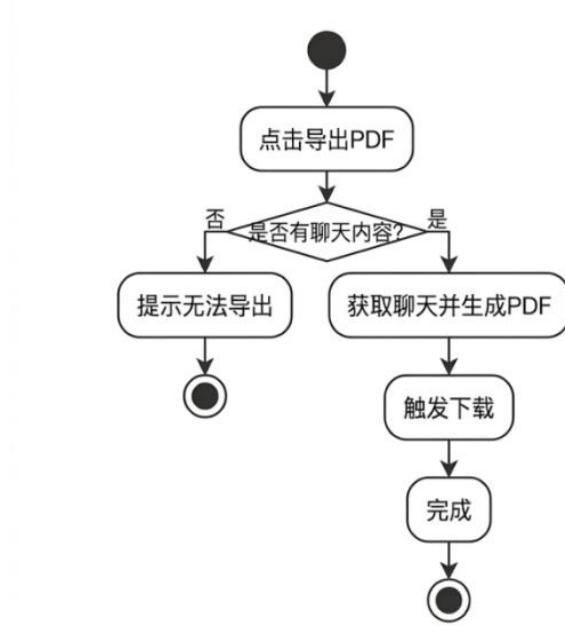


图 4-10 会话导出流程活动图

(3) 对话收藏功能设计

当用户在前端主聊天界面或历史会话中查阅对话时，可以针对有价值的回复进行收藏。如图 4-11 所示，收藏操作是一个从用户点击到后端持久化，再到前端统一呈现的闭环流程。后端通过校验 消息的 id 防止重复收藏，并冗余存储了具体的 问题和答案文本，确保即使原会话被删除，关键的收藏内容依然可用且独立。收藏后用户可进入收藏夹中查看曾经收藏过的内容，同时也可以取消收藏。

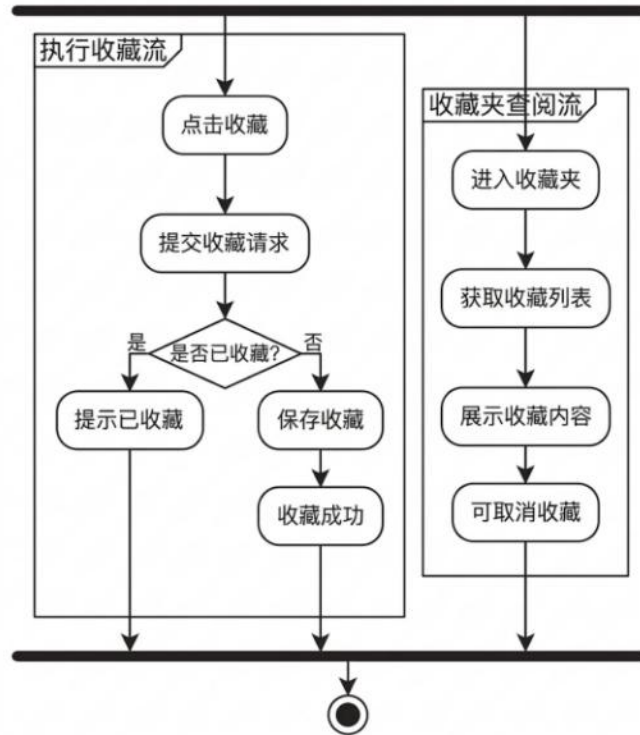


图 4-11 对话收藏与查看活动图

4.4 数据库设计

本系统在数据库选型上采用了“关系型+向量型”的双模型数据库架构。其中，针对用户信息、对话记录、反馈等结构化业务流转数据，系统选用了轻量级的关系型数据库 SQLite；针对非结构化的法律文献、条款原文等文本数据，系统选用了专为大语言模型打造的向量数据库 ChromaDB，用于存储切片后文本的高维向量特征。由于向量数据库主要负责隐式检索，因此本节及下文的实体属性与表结构设计，将主要围绕 SQLite 关系型数据库中的各个核心业务数据表展开说明。

4.4.1 核心功能表

核心功能部分主要围绕系统用户的身份记录及多轮对话上下文数据展开。这部分实体主要包含用户 (users)、对话记录 (conversations) 和消息明细 (messages)。由于多轮对话隔离的需求，系统规定一个用户可以发起多个独立的对话，而每个对话中又包含连续往复的多条消息明细，三者构成了标准的一对多层级关联模型。其核心功能表的 E-R 图如图 4-12 所示：

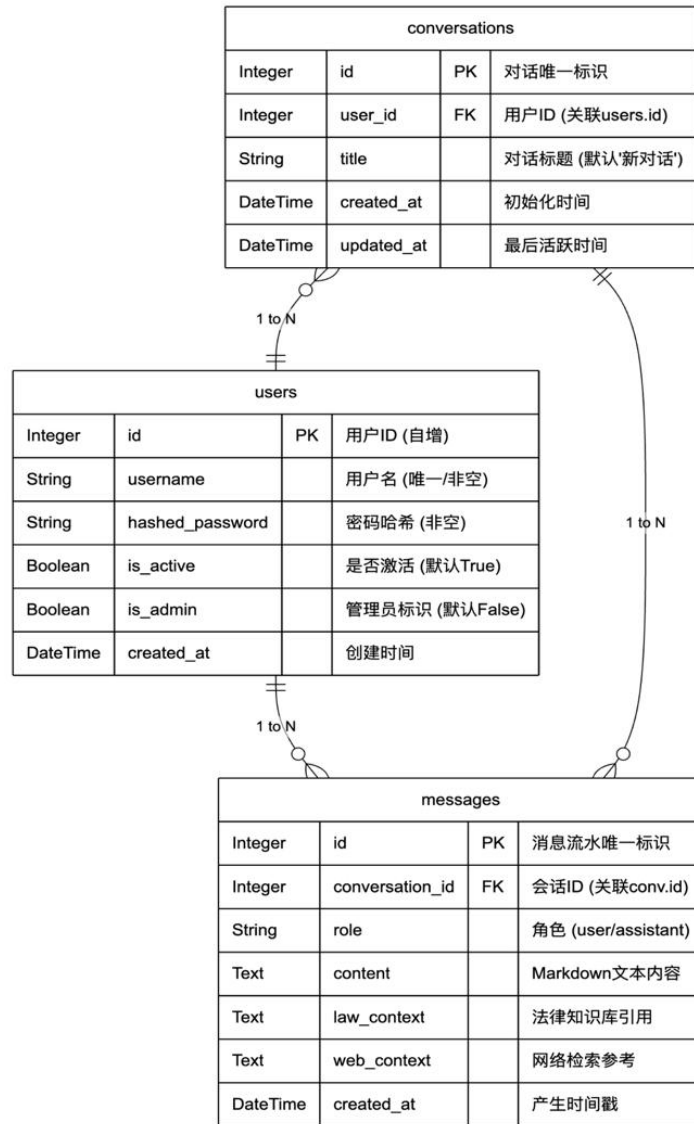


图 4-12 核心功能 E-R 图

(1) 用户表 (users) :

用户表主要是用于存储的系统用户的基本信息和登录认证的相关数据，包含 id、username、hashed_password、is_active、is_admin 和 created_at 字段，其中 id 是主键。该表也采用哈希 (hashed_password) 这一个处理密码信息的方式来对用

户隐私密码进行加强。同时通过 `is_admin` 字段这个字段来区分普通的用户和管理员。用户表的详细字段信息如表 4-5 所示。

表 4-5 用户表

字段名	类型	约束	描述
id	Integer	主键, 自增	用户的唯一标识 (系统分配)
username	String(50)	唯一, 非空, 普通索引	用户名 (登录凭据)
hashed_password	String(255)	非空	密码哈希值 (单向不可逆加密)
is_active	Boolean	默认 True	账户是否被激活可用
is_admin	Boolean	默认 False	管理员核心标识 (用于权限控制路由)
created_at	DateTime	默认当前时间	账户创建注册时间

(2) 对话记录表 (conversations) :

对话记录表用于切分和管理用户独立的多轮对话上下文, 主要包括 `id`、`user_id`、`title`、`created_at` 和 `updated_at` 字段, 其中 `id` 是主键。其中每个用户可以创建大量的无状态并列对话模型 (`user_id`), 同时通过保留 `updated_at` 字段, 方便前端系统实现以时间倒序的活跃度历史排序展示。对话记录表的详细字段信息如表 4-6 所示。

表 4-6 对话记录表

字段名	类型	约束	描述
id	Integer	主键, 自增	对话的唯一标识
user_id	Integer	外键(users.id), 非空	该对话所属的用户 ID
title	String(200)	默认 "新对话"	对话列表页显示的概要/标题
created_at	DateTime	默认当前时间	对话树初始化创建的时间

updated_at	DateTime	ON UPDATE 当前时间	对话最后活跃的更新时间（自动维护）
------------	----------	----------------	-------------------

(3) 消息明细表 (messages) :

消息明细表用于存放所有实际的双边问答日志以及大模型引用的上下文, 主要包括 id、conversation_id、role、content、law_context、web_context 和 created_at 字段, 其中 id 是主键。其中拓展了 law_context (法律证据依据) 和 web_context (网络引用源) 两个长文本字段, 这规定了每次 RAG 生成时使用的具体上下文, 防止模型“幻觉”。消息明细表的详细字段信息如表 4-7 所示。

表 4-7 消息明细表

字段名	类型	约束	描述
id	Integer	主键, 自增	消息流水的唯一标识
conversation_id	Integer	外键 (conversations.id), 非空	级联绑定的会话 ID
role	String(20)	非空	角色指代 ("user" 用户 或 "assistant" 模型)
content	Text	非空	发送和回复的具体 Markdown 文本内容
law_context	Text	允许为空	回复时溯源引用的法律条文/内部知识库原文
web_context	Text	允许为空	联网检索补充的 DuckDuckGo 网页参考来源
created_at	DateTime	默认当前时间	当前问/答产生的物理时间戳

4.4.2 辅助功能与后台统计表

辅助功能及统计部分着眼于满足用户的个性化标记需求,同时协助后台管理者把控系统质量。这部分涉及收藏夹表(favorites)、问题日志审计表(question_logs)与用户反馈表(feedbacks)。在业务联系上,所有的动作源头均关联至特定用户(user_id),其中收藏夹和主观反馈更与特定的历史消息(message_id)做直接映射,以构筑完善的人工纠错与复用机制。辅助表、后台统计表与核心功能表的关联的 E-R 图如图 4-13 所示:

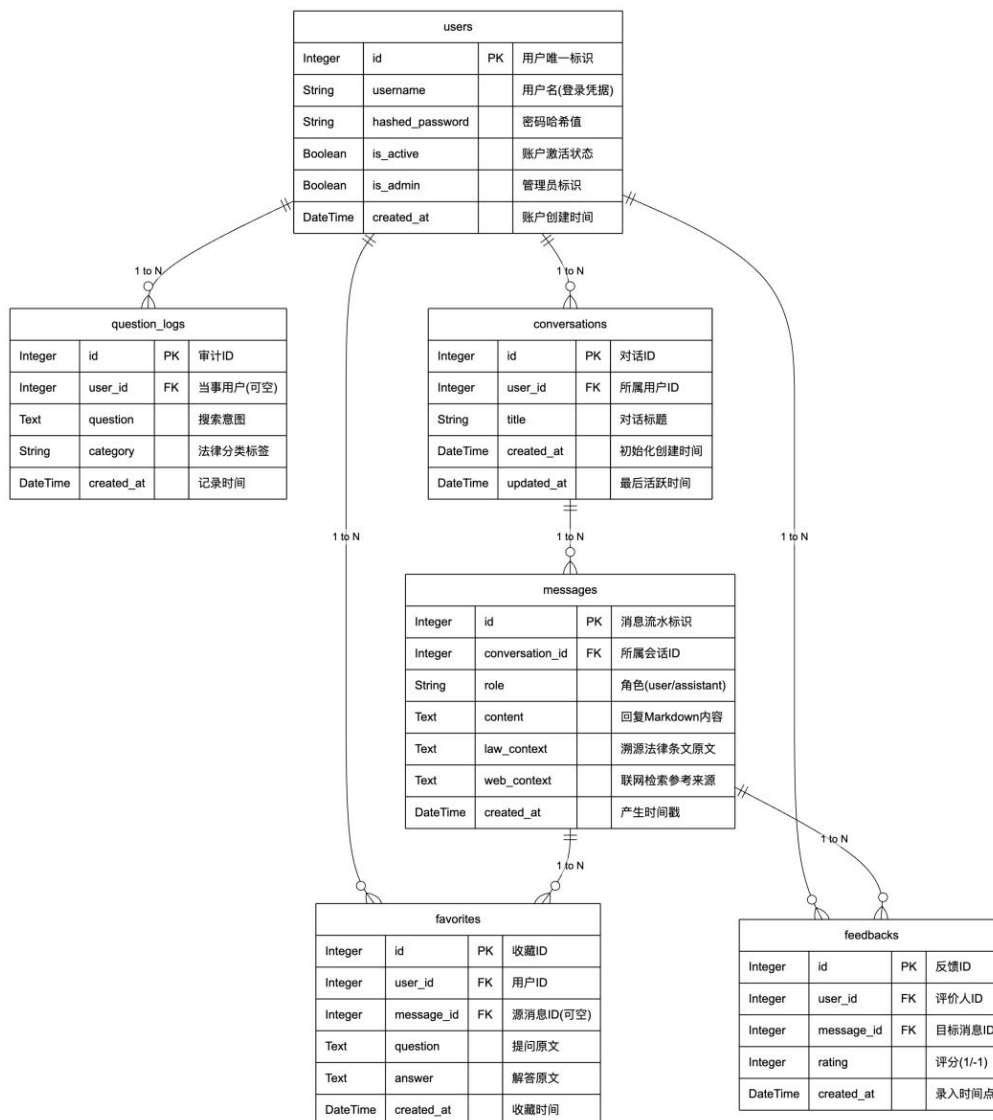


图 4-13 辅助、统计功能表与核心功能表的 E-R 图

(1) 收藏夹表 (favorites):

收藏夹表是用于为用户永久保存带有高意向价值的问题与解答记录,主要包括 id、user_id、message_id、question、answer 和 created_at 字段,其中 id 是

主键。收藏夹表的详细字段信息如表 4-8 所示。

表 4-8 收藏夹表

字段名	类型	约束	描述
id	Integer	主键, 自增	用户收藏条目的唯一标识
user_id	Integer	外键(users.id), 非空	触发保存动作的用户 ID
message_id	Integer	外键(messages.id), 可空	被收藏的源消息节点 (允许级联失效时留空)
question	Text	非空	收藏时的提问原文
answer	Text	非空	收藏时的 AI 诊断解答原文
created_at	DateTime	默认当前时间	加入收藏夹的精确时间戳

(2) 问题日志审计表(question_logs):

问题日志审计表用于帮助管理员在监控面板做统计绘图及分析用户的搜索倾向, 主要包括 id、user_id、question、category 和 created_at 字段, 其中 id 是主键。此表是预留给管理员监控面板做 ECharts 绘图提取统计用表。将用户的原始输入内容与类型标记 (category) 集中管理, 便于后端分析当前系统的“领域热搜”法律倾向。问题日志审计表的详细字段信息如表 4-9 所示。

表 4-9 问题日志审计表

字段名	类型	约束	描述
id	Integer	主键, 自增	审计记录的唯一主键 ID
user_id	Integer	外键(users.id), 允许为空	提出搜索访问的当前用户
question	Text	非空	抽离出的搜索文本意图

category	String(50)	允许为空	依靠分类算法打上的核心标签（民法/刑法等）
created_at	DateTime	默认当前时间	会话流经管道的采集记录时间

(3) 用户主观反馈表 (feedbacks):

用户主观反馈表用于收集用户的人工质量反馈标注以改善系统后续的回答质量，主要包括 id、user_id、message_id、rating 和 created_at 字段，其中 id 是主键。由于大语言模型回答问题的局限性，此表建立的核心目的是为了构建人工反馈标注。管理员可将大量呈现评分为 -1 的差评问答从本表中筛出提取特征，以此改善系统回答。用户主观反馈表的详细字段信息如表 4-10 所示。

表 4-10 用户主观反馈表

字段名	类型	约束	描述
id	Integer	主键，自增	评价收集记录的唯一标记 ID
user_id	Integer	外键(users.id)，非空	在前端大拇指上点击动作的 ID 追踪源
message_id	Integer	外键(messages.id)，非空	被用户标注具体判定的系统 AI 返回记录
rating	Integer	非空	1 代表满意(推荐) / -1 代表不合格(糟糕)
created_at	DateTime	默认当前时间	评阅与打分产生的录入时间点

4.5 本章小结

本章主要对系统的整体设计与实现思路进行了说明。我们先介绍了系统的整体架构，包括前端 Vue3、后端 FastAPI、大语言模型以及向量数据库之间是如何配合完成法律问答的。随后，围绕法律咨询场景，对系统中的一些关键流程进行了设计，例如问题是否属于法律问题的判断、用户问题重写、法律知识检索以及最终回答生成等内容。同时，为了让系统回答更加稳定，我们也对法律文档切分、提示词设计以及多轮对话处理进行了说明，并加入了收藏、PDF 导出等一些实际使用中比较常见的功能。在数据存储部分，本章还介绍了数据库表的基本设计方式。总体来看，本章完成了系统整体方案与主要功能的设计，也为后续系统实现与功能测试做好了准备。

第 5 章 系统实现

5.1 系统登录与注册功能实现

系统在登录页提供“登录/注册”双 Tab 交互。用户注册成功后可直接切换登录；登录成功后自动跳转系统主页并加载当前用户信息。输入账号密码后点击登陆后即可进入问答系统，如图 5-1，图 5-2 所示。

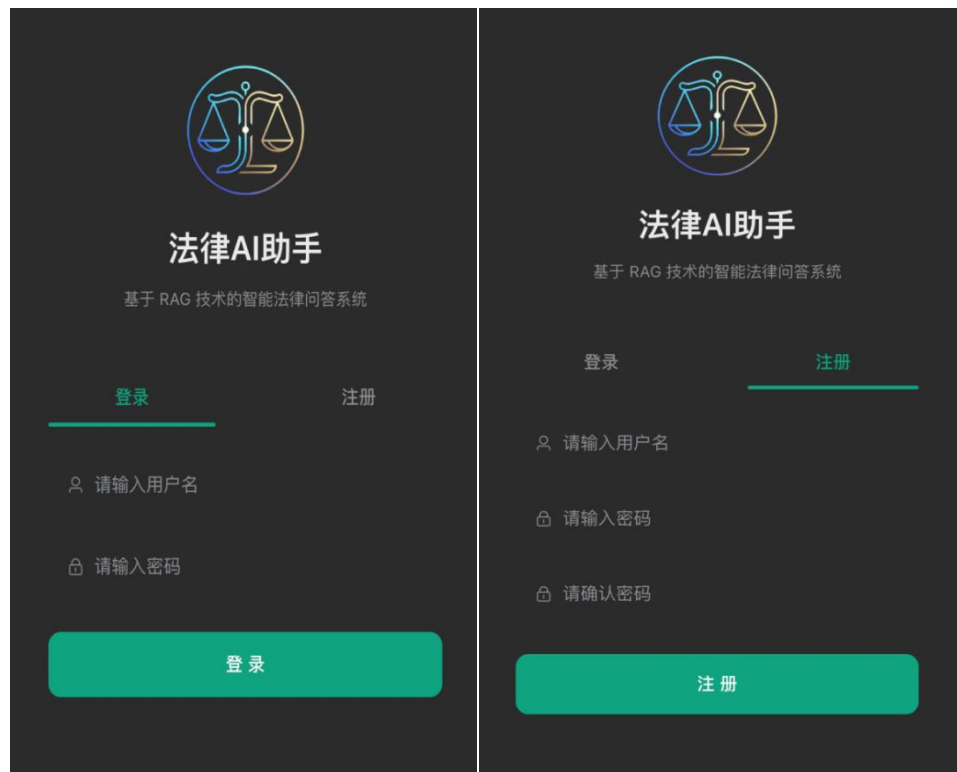


图 5-1 系统登录注册界面

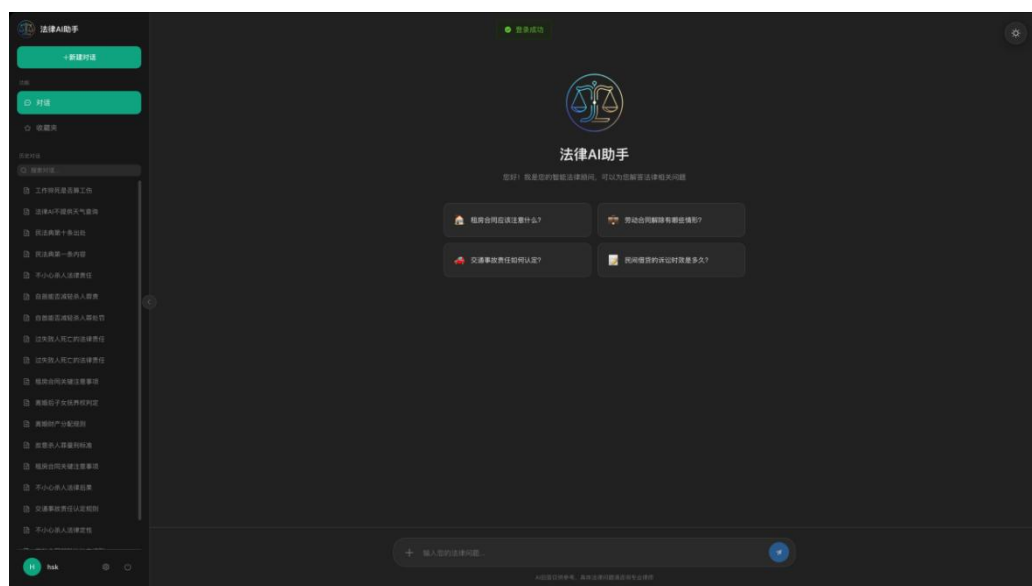


图 5-2 问答系统界面

注册与登陆界面的核心代码如下所示：

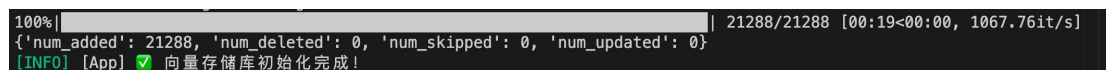
```
# backend/routers/auth.py
@router.post("/register", response_model=UserResponse)
async def register(user_data: UserCreate, db: Session = Depends(get_db)):
    existing_user = get_user_by_username(db, user_data.username)
    if existing_user:
        raise HTTPException(status_code=400, detail="用户名已被注册")
    user = create_user(db, user_data.username, user_data.password)
    return user

@router.post("/login", response_model=Token)
async def login(form_data: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)):
    user = authenticate_user(db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(status_code=401, detail="用户名或密码错误")
    access_token = create_access_token(data={"sub": user.username, "user_id": user.id})
    return {"access_token": access_token, "token_type": "bearer"}
```

代码里面的这个注册接口主要完成用户名检查和账户创建两个步骤。系统会先判断用户名是否已被使用，如果已存在，则不给人再重复注册；如果不存在，则创建新账户。同时，密码不会以明文形式保存，而是经过处理后再存入数据库。登录接口则主要负责账号验证。用户输入用户名和密码后，系统会先进行匹配检查，验证成功后生成登录令牌，再返回给前端保存，用于后续请求时的身份识别。

5.2 知识库管理模块实现

在部署或知识库更新时，执行向量库构建脚本后，会依次进行加载法律文书、文本切分、向量化入库。若使用 `--clear` 参数，系统会先清空旧向量再重建。如图 5-3 所示



```
100% | 21288/21288 [00:19<00:00, 1067.76it/s]
{'num_added': 21288, 'num_deleted': 0, 'num_skipped': 0, 'num_updated': 0}
[INFO] [App] ✓ 向量存储库初始化完成！
```

图 5-3 向量数据库构建图

其核心代码如下所示：

```
# scripts/build_vector_db.py
loader = LawLoader(law_dir)
documents = loader.load()
splitter = LawSplitter(chunk_size=chunk_size, chunk_overlap=chunk_overlap)
split_docs = splitter.split_documents(documents)
```

```

result = law_index(split_docs, show_progress=True)
# law_ai/splitter.py
class LawSplitter(RecursiveCharacterTextSplitter):
    def __init__(self, **kwargs: Any) -> None:
        separators = [r"第\S*条 "]
        is_separator_regex = True
        headers_to_split_on = [("#", "header1"), ("##", "header2"), ("###",
"header3"),(####, "header4"),]
        self.md_splitter =
MarkdownHeaderTextSplitter(headers_to_split_on=headers_to_split_on)
        super().__init__(separators=separators, is_separator_regex=is_separator_regex,
**kwargs)
# law_ai/utils.py
def get_vectorstore(collection_name: str = "law") -> Chroma:
    return Chroma(
        persist_directory="./chroma_db",
        embedding_function=get_cached_embedder(),
        collection_name=collection_name)
def law_index(docs: List[Document], show_progress: bool = True) -> Dict:
    record_manager = get_record_manager("law")
    vectorstore = get_vectorstore("law")
    ...
    result = index(batch, record_manager, vectorstore, cleanup=None,
source_id_key="source")

```

其中这个 build_vector_db.py 主要是来负责法律知识库的构建。系统会先读取本地法律文档内容，又对文本进行切分，再完成向量化处理，最后保存到向量数据库中。

在文本切分部分，系统使用了 LawSplitter 对法律文档进行处理。因为法律文本通常都是比较长的，而且存在章节与法条的划分，因此系统不会直接把整篇文档一次性处理，是先按照 Markdown 标题进行简单划分，再结合“第 X 条”等形式来拆分内容。这样处理后，每段文本的内容会相对就完整一些了，也就更方便后面进行检索。

在数据保存部分，系统会把处理后的法律文本转换成向量，并写入 Chroma 数据库中。相关数据会保存在 chroma_db 目录下。

5.3 智能法律问答模块实现

当用户输入法律问题后，系统会根据问题内容生成对应回答，并同时展示相关法律依据以及网络检索信息，方便用户查看回答所参考的内容。整体展示效果如图 5-4 与图 5-5 所示。

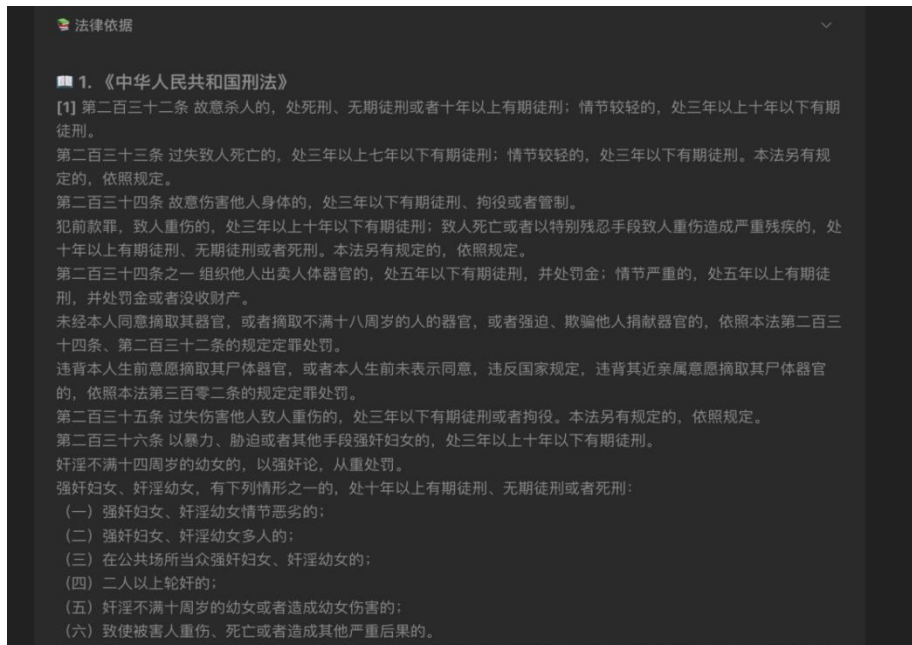


图 5-4 法律依据图

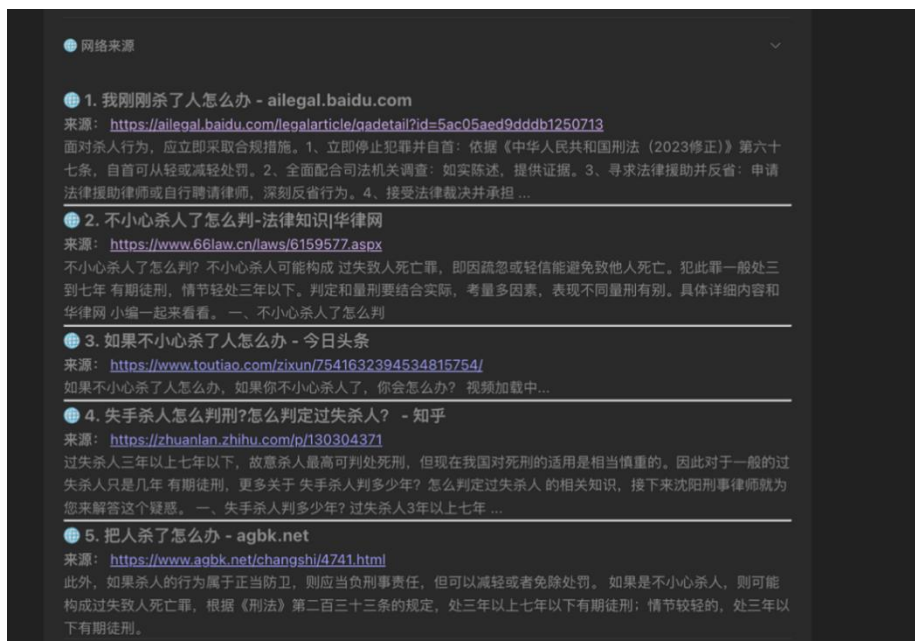


图 5-5 网络依据图

在回答过程中，系统采用流式输出的方式，生成内容会逐步显示在页面中，用户不用等全部话生成完成就可以看到了。回答结束后，当前对话内容会自动保存到历史记录中。其效果如图 5-6 所示。

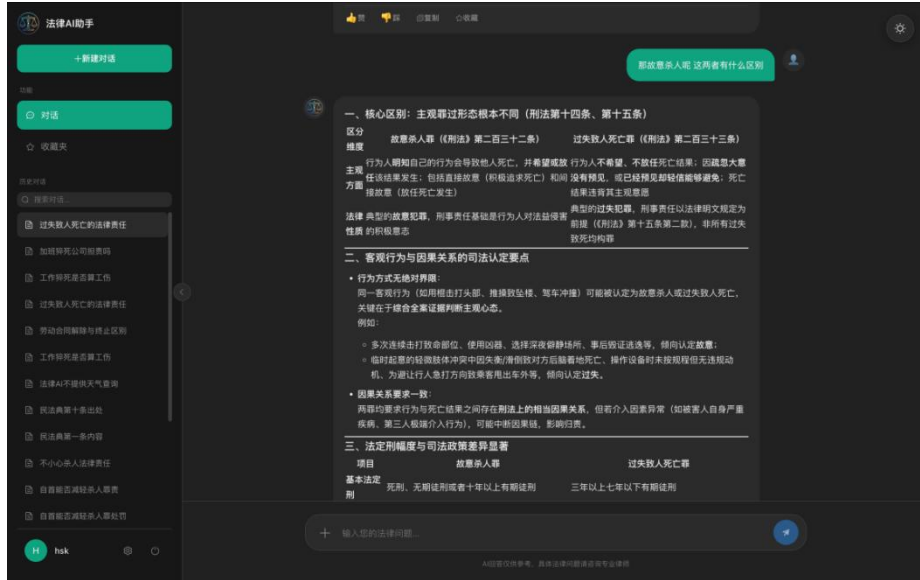


图 5-6 历史上下文回答图

其核心代码如下所示：

```
# law_ai/chain.py
chain = ( RunnableMap( {
    "law_docs": lambda x:
multi_query_retriever.invoke(enhance_search_with_document(x)),
    "web_docs": lambda x:
web_retriever.invoke(enhance_search_with_document(x)),
    "question": lambda x: x["question"],
    "history": lambda x: x.get("history"),
    "uploaded_document": lambda x: x.get("uploaded_document")}
) | RunnableMap( {
    "law_context": lambda x: combine_law_docs(x["law_docs"]),
    "web_context": lambda x: combine_web_docs(x["web_docs"]),
    "question": lambda x: x["question"],
    "history": lambda x: x.get("history"),
    "uploaded_document": lambda x: x.get("uploaded_document")}
) | RunnableMap( {
    "answer": log_prompt_and_call_llm,
    "law_context": lambda x: x["law_context"],
    "web_context": lambda x: x["web_context"]}
) )
# law_ai/retriever.py
```


其核心代码如下所示：

```
# backend/routers/chat.py
@router.post("/stream", summary="流式发送消息")
async def chat_stream(...):
    async def generate(): full_answer = "" law_ctx = "" web_ctx = ""
        async for chunk in law_qa_service.stream_answer(...):
            if isinstance(chunk, dict):
                law_ctx = chunk.get("law_context", "")
                web_ctx = chunk.get("web_context", "")
            else: full_answer += chunk yield f"data: {json.dumps({'token': chunk},
ensure_ascii=False)}\n\n"
        yield f"data: {json.dumps({'done': True, 'conversation_id': conversation_id,
'law_context': law_ctx, 'web_context': web_ctx}, ensure_ascii=False)}\n\n"
    return StreamingResponse(generate(), media_type="text/event-stream")
// frontend-user/src/stores/chat.js
for await (const chunk of api.chatStream(enhancedQuestion, this.currentConversationId,
documentId)) {if (chunk.token) { fullContent += chunk.token
    this.messages[aiMessageIndex].content = fullContent } else if (chunk.done) {
    lawContext = chunk.law_context || " webContext = chunk.web_context || "}}
```

系统在回答问题时采用了流式返回的方式。也就是说，后端不会等模型把完整回答全部生成后再一次性返回，而是在生成过程中不断把内容发送给前端，因此用户可以一边等待，一边看到回答逐渐显示出来。

在后端部分，系统通过 `StreamingResponse` 这个函数来持续的返回生成内容。当模型开始输出回答后，系统会实时把生成的文字发送到前端页面；当回答全部结束后，再返回当前会话编号以及相关法律法规和网络参考内容。

前端接收到内容后，会不断更新聊天区域中的消息显示，因此回答内容会逐步出现在页面中，而不是一次性全部展示。然后，为了便于后续继续咨询，系统会在问答结束后自动保存当前聊天记录。这样用户再次进入历史会话时，可以继续围绕之前的问题进行追问，也方便查看过去的咨询内容。

考虑到法律咨询内容可能需要长期保留，系统还增加了收藏与导出功能。用户可以在回答区域点击“收藏”这个按钮，将重要内容保存下来，并在收藏列表中再次查看；同时，也支持将历史会话导出为 PDF 文件，方便保存或后续查阅。相关效果如图 5-8 与图 5-9 所示。



图 5-8 对话 pdf 导出图



图 5-9 收藏夹效果图

核心代码如下所示：

```
# backend/routers/favorites.py
@router.post("", response_model=FavoriteResponse)
async def add_favorite(data: FavoriteCreate, ...):
    favorite = Favorite(
        user_id=current_user.id,
        message_id=data.message_id,
        question=data.question,
        answer=data.answer
    )
    db.add(favorite)
    db.commit()// frontend-user/src/views/Layout.vue
```

```
const exportConversationToPDF = async (conversationId) => {  
  const res = await api.getMessages(conversationId)  
  const messages = res.messages || []  
  const opt = {  
    margin: 10,  
    filename: `${title}.pdf`,  
    html2canvas: { scale: 2, useCORS: true },  
    jsPDF: { unit: 'mm', format: 'a4', orientation: 'portrait' }  
  }  
  await html2pdf().set(opt).from(content).save()  
}
```

从功能实现来看,系统增加了收藏和 PDF 导出两个功能,方便用户保存重要的法律咨询内容。在收藏部分,当用户点击收藏按钮后,系统会把当前问答内容保存下来,后续可以在收藏页面再次查看。这样即使聊天记录比较多,用户也能更快找到之前的重要内容。

在 PDF 导出部分,系统会读取当前会话中的聊天记录,并生成对应的 PDF 文件。导出的内容主要来自系统中的问答记录,因此用户在页面中看到的内容,也基本能够保留在导出的文件中,方便后续查看和保存。

5.5 管理员后台统计可视化功能实现

管理员登录后可查看系统总览、用户增长、问题趋势、满意度、Token 消耗及用户管理页面,实现运行态监控与运营分析。如图 5-10 和图 5-11 所示:



ID	用户名	状态	角色	对话数	提问数	注册时间	操作
4	test1	正常	用户	5	5	2026/01/18 09:01	禁用 重置密码
3	mql	正常	用户	0	0	2026/01/18 06:49	禁用 重置密码
2	hsk	正常	用户	103	142	2026/01/07 09:41	禁用 重置密码
1	admin	正常	管理员	0	0	2026/01/07 09:40	禁用 重置密码

图 5-10 用户管理页面



图 5-11 系统后台统计总览

其核心代码如下所示：

```
# backend/routers/admin.py
@router.get("/stats", response_model=AdminStatsResponse)
async def get_admin_stats(...):
    total_users = db.query(User).count()
    total_conversations = db.query(Conversation).count()
    total_messages = db.query(Message).count()
    total_tokens = db.query(func.coalesce(func.sum(TokenUsage.total_tokens), 0))
    .scalar()
    today_tokens = db.query(func.coalesce(func.sum(TokenUsage.total_tokens), 0))
    .filter(
        func.date(TokenUsage.created_at) == today_str).scalar()
    return { "total_users": total_users, "total_conversations": total_conversations,
"total_messages": total_messages, "total_tokens": total_tokens, "today_tokens": today_tokens,
...}

// frontend-admin/src/views/AdminDashboard.vue
const initTokenUsageChart = () => {
    const data = stats.value?.token_trend || []
    chart.setOption({ series: [
        { name: 'Prompt Tokens', type: 'bar', stack: 'tokens', data: data.map(d =>
d.prompt_tokens) },
        { name: 'Completion Tokens', type: 'bar', stack: 'tokens', data: data.map(d =>
d.completion_tokens) }]} }) }
```

这段代码给系统增加了后台统计功能，方便管理员查看系统的基本使用情况。系统会统计当前用户数量、聊天会话数量、消息数量以及模型调用时产生的Token 数据，并统一展示在后台页面中。

在数据处理部分，后端会从数据库中读取相关内容，并计算总使用量以及当天的使用情况。管理员进入后台后，可以直接查看这些统计结果，不需要再单独查询数据库。

在页面展示部分，系统使用图表对部分数据进行显示。例如，Token 使用情况会按照时间变化进行展示，使数据看起来更加清楚，也方便管理员查看近期使用情况。

5.6 本章小结

本章主要介绍了系统是如何一步一步实现出来的。我们先说明了用户注册、登录等基础功能的实现方式，保证用户能够正常进入系统并使用法律咨询功能。之后，又介绍了法律知识库的构建过程。在法律问答部分，本章重点展示了系统是如何根据用户问题查找相关法律内容。同时，为了让用户使用起来更方便，我们还实现了流式回答、历史记录保存、收藏以及 PDF 导出等功能。此外，本章还简单介绍了后台管理模块的实现，包括用户数量、聊天记录以及模型使用情况等统计功能。总体来看，到这一章为止，已经把系统的大部分核心功能完成，并能够支持基本的法律咨询流程。

第 6 章 系统测试

6.1 测试方案设计

本章主要说明我们是如何对系统进行测试的。测试的重点是看看系统能不能正常使用，以及回答效果和运行速度是否符合预期。在功能方面，我们主要测试了用户登录注册、法律问答、多轮对话、收藏与导出，以及后台统计这些功能是否能正常运行，确保整个系统的主要功能都可以顺利使用。在回答效果方面，我们对比了使用 RAG 和不使用 RAG 的两种情况，看看加入法律知识检索之后，回答是否更准确，引用的法律内容是否更可靠，用来判断系统在减少错误回答方面的表现。在性能方面，我们主要关注系统回答问题的速度，以及在连续使用时是否会出现卡顿或不稳定的情况。为了保证测试结果尽量真实，本章的测试环境统一在 macOS 系统下完成，使用 Python 3.11 作为后端运行环境，前端用了 Vue3 和 Element Plus，数据库使用 SQLite 和 Chroma，大模型使用 Qwen。

6.2 功能测试结果

为了检查系统各个功能是否都能正常使用，本次测试把主要功能都跑了一遍，包括法律问答、多轮对话、用户登录、收藏、PDF 导出，以及后台统计等模块。

测试过程中，主要是按照实际使用流程来操作系统，比如先登录账号，然后进行提问，再查看回答效果，以及测试收藏和导出功能是否正常。如表 6-1 所示。

从结果来看，法律相关问题可以正常识别，不属于法律的问题会被拦截；在多轮对话中，系统也能根据上下文继续回答，不会出现明显断裂的情况。在账号登录方面，可以正常完成注册、登录，并且接口权限控制是正常的，请求需要携带 Token 才能访问。收藏功能可以把重要问答保存下来，重复收藏时系统也会给出提示。导出 PDF 功能也可以正常生成文件，内容顺序和页面展示基本一致。整体来看，这些功能一起使用时没有出现明显问题，基本可以满足日常使用需求。

表 6-1 功能测试表

测试项	关键验证点	结果
法律文档向量化与知识库构建	能够正确解析法律文档并进行段落切分，成功调用 Embedding 模型计算向量并存入数据库	通过
法律相关性判断	宽泛问题在无上下文时拒答；有法律上下文时可继续对话	通过
代词问题重写	“这两种/那个/它们”等指代问题可重写为明确检索问题	通过
多轮连续对话	上下文延续、话题切换与历史消息利用	通过
用户登录与鉴权	登录成功后返回 Token；鉴权接口可识别当前用户身份	通过
对话收藏功能	消息可加入收藏；重复收藏触发去重提示；收藏列表可见	通过
对话导出功能	历史会话可导出 PDF；导出内容与页面问答一致	通过
管理员登录与数据看板	管理员登录跳转、统计面板加载与接口访问	通过

为了更直观地展示上述测试项的具体执行细节与场景边界，并便于后续其他人员复现实验过程，我们在测试集中抽取了部分典型场景的测试数据。这些验证用例详细规定了前置条件、操作输入，以及用于比对的预期表现。表 6-2 给出了核心功能的代表性测试样例。

表 6-2 功能测试用例表

核心功能	前置条件	输入/操作	预期结果	实际结果
法律文档向量化与知识库构建	准备包含法律条款的文档	解析文档，切分后并向量化入库知	文档被正确切分段落，成功生成对应 Embedding 并持久化至向量数据库中	通过

法律相关性判断	清空会话上下文	提问：“今天天气怎么样？”	系统判定为非法律问题并给出拒答提示	通过
法律语境延续判断	历史消息包含“故意犯罪与过失犯罪区别”	追问：“这两者有什么区别？”	系统识别上下文中的“这两者”，继续法律问答	通过
代词问题重写	历史消息包含“劳动合同解除与终止”	追问：“这两种情形赔偿一样吗？”	系统将代词问题重写为可检索的明确问题	通过
多轮连续对话	建立四轮会话	第 1-3 轮围绕同一法律主题，第 4 轮切换“劳动争议仲裁时效”	前三轮上下文连贯；第四轮成功完成话题切换	通过
用户登录与鉴权	已创建普通用户账号	输入正确账号密码登录后访问鉴权接口	登录成功返回 Token，接口返回当前用户信息	通过
对话收藏功能	用户已登录并存在至少 1 条 AI 回复	点击消息“收藏”按钮后，再次点击同一消息收藏	首次收藏成功并出现在收藏列表；再次收藏提示已收藏	通过
对话导出功能	用户已登录并存在历史会话消息	在历史会话下拉菜单点击“导出 PDF”	成功下载 PDF，内容覆盖完整问答且顺序正确	通过
管理员登录与看板	已创建管理员账号	管理员登录后访问统计看板页面	登录成功，统计卡片与趋势图正常加载	通过

6.3 效果评估

6.3.1 回答对比

本节采用同一批 50 个法律问题，对两种回答方式进行对照测试：一组仅使用模型直接回答（Baseline），另一组在回答前引入检索增强（RAG）。两组使

用相同模型、相同问题集合和统一评分口径，重点观察四个方面：回答质量是否提升、法律依据是否更充分、响应速度是否可接受、输出内容长度是否可控。为避免单一评分偏差，本节同时使用自动评分、专家模型评分与综合评分，并结合逐题对比结果进行分析。

评分执行方式如下：自动评分由评测程序按统一规则自动计算；专家模型评分由独立评审模型完成；综合评分由系统将规则评分、专家模型评分和成对比较评分进行加权融合后得到。其核心指标如表 6-3 所示。

表 6-3 核心指标对比表

指标	Baseline（直接回答）	RAG（检索增强）	差值 (RAG-Baseline)
平均自动评分	33.28	55.94	+22.66
平均综合评分	59.10	73.71	+14.61
平均专家模型评分	89.76	93.10	+3.34
平均时延（ms）	27511.72	28668.23	+1156.51
成本代理（平均输出字符）	1773.64	1492.26	-281.38

从总体结果看，检索增强方案在三类质量指标上均优于直接回答方案，尤其在综合评分上提升明显，说明其在法律问答场景中的稳定增益较强；与此同时，检索增强带来约 1.16 秒的平均时延增加，体现出“质量提升与响应速度”之间的工程权衡。

其中，三个核心质量指标的含义如下：

(1) 平均自动评分：指每道题由规则引擎自动打分后的平均值，主要依据关键词命中、法律名称命中、引用与证据一致性、回答与证据贴合度、是否拒答等客观信号。

(2) 平均综合评分：指每道题的最终得分平均值，该最终得分由规则评分、专家模型评分加权融合得到，用于反映整体答复质量。

(3) 平均专家模型评分：指独立评审模型对每道题进行专业评议后的平均值，重点考察内容相关性、事实与法律真实性、答非所问程度以及法律引用质量。

6.3.2 引用准确率评估

本节用于评估系统回答中法律引用的可追溯性与可信度。测试对象为评测过程中产生的 AI 回答消息，测试流程分为三个步骤：首先从回答文本中抽取法律名称与条文引用；其次在该回答对应的检索证据中进行逐条回查，判断该引用是否可被证据支持；最后汇总统计总引用数、已验证引用数、未验证引用数，并计算整体准确率。其测试结果如表 6-4 所示。

表 6-4 引用准确率测试表

指标	数值
AI 消息数	95
总引用数	1125
已验证引用数	1070
未验证引用数	55
总体准确率	92.9%

从测试结果来看，在 95 条回答中，一共生成了 1125 条引用，其中 1070 条可以在检索结果中找到对应依据，引用准确率达到 92.9%，55 条暂时无法完全匹配，未验证引用占比约 7.1%，仍存在进一步优化空间。

整体来看，大部分引用是可以对应到原始法律内容的，说明系统在生成回答时基本能够参考到已有的检索结果。不过也有一小部分引用没有完全匹配到对应证据，这部分占比不大，但说明在某些问题场景下检索结果还不够完整，或者模型在表达时出现了一定偏差。总体来说，系统在引用一致性方面表现还可以，但在少数情况下仍然会出现无法完全对应的情况，后面可以再针对检索范围和提示词做一些调整。

6.4 非功能测试结果

6.4.1 性能测试

在系统的法律智能问答服务中，回答的时延响应是评估 RAG 系统落地可行性的关键。本次性能测试着重模拟了真实的高频咨询场景的任务。性能测试的样本库选取了覆盖《刑法》、《道路交通安全法》等多个领域的 50 个高频法律提问模型与标准未使用 RAG 的大模型进行对比测试。主要采用自动化脚本记录响应时延（Latency）、批量执行的完成率。性能测试结果如表 6-5 所示：

表 6-5 性能测试表

用例编号	测试项	度量与触发方式	预期结果	实际结果	测试状态
TC-PERF-001	回答时延控制	平均响应时延对比 (Baseline VS 检索增强 RAG)	平均时延控制在 30s 以内，且引入 RAG 后的时延相对增幅 $\leq 2s$ 。	平均时延由 27.51s 增至 28.67s，绝对增幅仅为 1.16s。	通过
TC-PERF-002	批量评测完成率	构造 50 个并发样本执行成功率测试	全部批量样本均可完成自动化评测，无因性能瓶颈导致的中断。	50/50 样本全部执行完成，输出的评测结果文件数据完整无缺漏。	通过
TC-PERF-003	批处理稳定性	压测环境下的异常中断与熔断重试机制检查	长评测过程无彻底中断，遇到网络或 API 异常抖动场景可自动恢复。	批处理过程全程无挂起，期间自动触发 1 次网络重试并成功恢复执行。	通过

结果表明，系统在响应效率方面满足预期，且批量执行过程具备较好的稳定性与异常恢复能力。

6.4.2 安全性测试

这一部分主要检查系统在安全方面是否能正常工作，重点看用户登录、数据保存和接口访问这些地方有没有明显问题。测试过程中，我们主要从几个常见情况入手，比如用户登录后的身份是否会失效、密码是否是加密存储的、接口在没有限权的情况下能不能访问，以及一些简单的注入请求是否会被拦住。在实际测

试中，登录后系统会生成 Token，用来判断用户身份，未携带 Token 的请求会被拒绝访问。数据库中的密码也不是明文存储，而是加密后的字符串。另外，在尝试一些异常请求时，比如带有非法 SQL 语句的输入，系统基本可以正常识别并拦截，不会直接影响正常功能使用。测试过程如表 6-6 所示。

表 6-6 安全性测试表

用例编号	测试项	测试与验证方式	预期结果	实际结果	测试状态
TC-SEC-001	登 录 密 码 加 密 存 储	数据库层面的账号数据物理安全防线检查	关系型数据库绝不保存明文密码，仅允许保存不可逆的加密哈希值。	经审查，用户密码 字 段 (password) 已采用哈希算法进行存储。	通过
TC-SEC-002	登 录 令 牌 安 全 保 护	身 份 令 牌 (Token) 签发机制	系统登录后发放带有明确过期时间的凭证令牌。	使用 JWT 签发 Token，同时包含过期声明。	通过
TC-SEC-003	关 系 库 SQL 注 入 防 护	登录认证的防注入代码检查	任何查询操作均应采用参数化查询或 ORM 条件构造，严禁字符串直接拼接 SQL。	认证逻辑与各类检索过滤均通过 ORM 框架条件约束实现，有效屏蔽非 SQL 拼接。	通过
TC-SEC-004	受 保 护 接 口 越 权 拦 截	对非登录态发起各种未授权 API 访问并进行拦截检查	携带无效令牌、过期令牌或无令牌的越权请求应被中间件无缝拒绝。	API 受保护路由器强依赖当前用户校验，在凭证无效、缺失时精准返回 HTTP,401/403 阻断响应。	通过

6.4.3 可维护性测试

为评估系统在后期运行过程中的可维护性与扩展能力，本文从数据更新机制、日志管理规范以及系统运行可追踪性等方面对系统进行了测试分析。可维护性测试重点关注系统在面对数据迭代、功能扩展及问题排查时的支持能力。

具体测试包括向量数据库的全量重建与增量更新能力验证，以及日志系统的规范性与业务链路追踪能力检查。通过对系统运维相关功能的验证，确保其在实际应用过程中具备良好的可扩展性与问题定位能力，从而降低后期维护成本。可维护性测试结果如表 6-7 所示。

表 6-7 可维护性测试表

用例编号	测试项	测试与验证方式	预期结果	实际结果	测试状态
TC-MNT-001	全量向量库重建能力	注入全新大规模法律文本后进行全量重建流程	法律文本迭代后，工程脚本可安全清空旧数据并重建	支持清空旧索引后进入全量重建	通过
TC-MNT-002	向量库增量更新能力	新增文档后进行增量构建流程检查	在不清空旧索引的前提下进行增量	支持默认增量构建流程，可按文档变化写入向量库	通过
TC-MNT-003	日志记录格式与规范	日志框架格式、隔离界限与留存策略检查	核心日志需标明时间戳、级别、所属模块边界，并提供安全持久化方案。	日志同时输出控制台与文件，按天滚动，保留最近 7 天	通过
TC-MNT-004	核心业务流追踪覆盖	关键业务日志覆盖检查	系统认证、链路跟踪及问答生成应具有完整的可追踪日志。	认证成功/失败等关键过程均有日志输出	通过

6.5 本章小结

本章主要是把系统整体跑了一遍测试，看看各个功能能不能正常用。我们先测试了基本功能，比如用户登录、问答、多轮对话、收藏、导出 PDF，还有后台统计这些模块，整体流程是可以正常跑通的，前后端配合也没有明显问题。在问答效果上，我们对比了一下有没有 RAG 的情况。结果是加了检索之后，回答会更贴近法律内容，也更容易给出对应的法律依据，不容易出现乱编的情况。在安全和性能方面，系统在常见的安全访问和数据处理上基本没有问题，只是在加了检索之后回答速度会稍微慢一点，但在法律问答这种场景下还能接受。总体来说，这个系统已经可以正常用来做基础的法律咨询，后面还有一些可以再优化的地方。

第7章 总结与展望

7.1 研究总结

本文主要做了一个面向法律咨询的问答系统，用检索增强生成的方法，让模型在回答问题时可以参考外部的法律资料，从而减少随意生成的情况。在实现过程中，我们把法律资料做了整理和结构化处理，让系统在回答问题时可以更容易找到对应的内容。同时在实际使用中，也加入了问题理解、信息补全和流式输出这些功能，使得用户在提问后可以比较顺畅地看到回答过程。测试情况来看，系统整体可以正常工作，在回答法律问题时会比直接用大模型更有依据一些，回答内容也更容易对应到具体的法律信息。总的来说，本研究不仅验证了结合外部知识库提升大模型专业能力的有效性，也为法律服务行业的智能化提供了可借鉴的技术参考。

7.2 未来展望

虽然目前这个系统已经可以完成基本的法律问答功能，但还有一些可以改进的地方。比如在一些比较复杂的问题中，检索结果有时候还不够精准，后面可以考虑优化检索方式，让相关内容找得更准一些。另外，部分法律条文的匹配结果还存在一定误差，这一块也还有优化空间。除此之外，后面也可以考虑支持更多类型的法律文书处理，比如合同或者判决书等。在表达方式上，也可以根据不同用户的理解水平，提供更简单或者更专业的回答方式，让不同人群都能更容易使用这个系统，从而进一步降低专业法律知识的获取门槛，推动智能法律咨询向更大众化的方向发展。

致 谢

首先衷心感谢我的指导老师王飞老师。从最初论文选题的确定，到系统架构的搭建，再到最后论文的反复修改，王老师都给予了我极大的支持。王飞老师待人随和，在指导过程中非常耐心，多次在关键部分提供指导，并给出切实可行的建议。王老师那种严谨、踏实的做事风格，让我深受启发。在此，向王老师的辛勤付出致以最诚挚的谢意。

同时，我要感谢电子与信息工程学院的所有授课老师。在大学四年的求知岁月中，各位老师始终保持着极其严谨的教学态度。正是老师们这种不遗余力、倾囊相授的付出，让我从零开始系统地掌握了计算机专业的基础知识。为我完成本系统的架构设计与逻辑实现打下了最坚实的基础。

感谢我的父母。在我忙于实验和写论文的日子里，是他们默默的理解与支持，让我能够全身心地投入到学业中，他们是最坚强的后盾。

最后，感谢苏州科技大学这四年的培养。尽管论文和系统还有很多不够成熟的地方，但这次完整的科研实践让我收获了宝贵的成长经验。也愿每一位奋斗在科技与法律融合道路上的同行者，都能不忘初心，砥砺前行。

参 考 文 献

- [1] 韩旭至.数字司法与人工智能治理的中国方案[J].华东政法大学学报,2025,28(01):5.
- [2] Xiao C, Hu X, Liu Z, et al. Lawformer: A pre-trained language model for chinese legal long documents[J]. AI Open, 2021, 2: 79-84.
- [3] 安震威,来雨轩,冯岩松.面向法律文书的自然语言理解[J].中文信息学报,2022,36(08):1-11.
- [4] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.
- [5] Ji Z, Lee N, Frieske R, et al. Survey of hallucination in natural language generation[J]. ACM computing surveys, 2023, 55(12): 1-38.
- [6] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks[J]. Advances in neural information processing systems, 2020, 33: 9459-9474.
- [7] Gao Y, Xiong Y, Gao X, et al. Retrieval-augmented generation for large language models: A survey[J]. arXiv preprint arXiv:2312.10997, 2023, 2(1): 32.
- [8] 林君,孙常龙,王琛鑫,等.大模型在法律应用中的探索与落地[J].法律与人工智能,2025,(01):224-237.
- [9] 王禄生.法律垂域大模型的存废之争、范式之议与能力之辨[J].法学论坛,2025,40(06):5-18.
- [10] 李明达,邸洪波,孙媛媛,等.基于法条检索的生成式法律问答研究[J].山西大学学报(自然科学版),2025,48(04):653-665.DOI:10.13451/j.sxu.ns.2024159.
- [11] HINDI M, et al. Enhancing the precision and interpretability of retrieval-augmented generation (RAG) in legal technology: a survey[J]. IEEE Access, 2025.
- [12] Cui J, Ning M, Li Z, et al. Chatlaw: A multi-agent collaborative legal assistant with knowledge graph enhanced mixture-of-experts large language model[J]. arXiv preprint arXiv:2306.16092, 2023.
- [13] Reimers N, Gurevych I S B. Sentence embeddings using siamese BERT-networks. arXiv 2019[J]. arXiv preprint arXiv:1908.10084, 1908, 10.
- [14] Han Y, Liu C, Wang P. A comprehensive survey on vector database: storage and retrieval technique[J]. Challenge, 2023: 1-13.
- [15] Pipitone N, LegalBench-RAG H A G. A benchmark for retrieval-augmented generation in the legal domain[J]. arXiv preprint arXiv:2408.10343, 2024.
- [16] Qiao S, Ou Y, Zhang N, et al. Reasoning with language model prompting: A survey[C]//Proceedings of the 61st annual meeting of the Association for Computational Linguistics (volume 1: long papers). 2023: 5368-5393.
- [17] Goebel R, Kano Y, Kim M Y, et al. Overview and discussion of the competition on legal information, extraction/entailment (coliee) 2023[J]. The Review of Socionetwork

- Strategies, 2024, 18(1): 27-47.
- [18] 王晓. 提示词工程实践[M]. 成都: 电子科技大学出版社, 2025.
- [19] Tsaneva S ,Dessi D ,Osborne F , et al.Knowledge graph validation by integrating LLMs and human-in-the-loop[J].Information Processing and Management, 025,62(5):104145-104145.
- [20] Bai J, Bai S, Chu Y, et al. Qwen technical report[EB/OL]. arXiv preprint, 2023. arXiv: 2309.16609.
- [21] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[J]. Advances in neural information processing systems, 2022, 35: 24824-24837.
- [22] Li Y, Chen T, Li Y, et al. 基于检索增强思维提示的汉语框架语义解析方法[C]//Proceedings of the 24th China National Conference on Computational Linguistics (CCL 2025). Jinan: Chinese Information Processing Society of China, 2025: 184-209.

附录 A 译文

附录 B 外文原文